Protocolos de Comunicaciones Industriales

Ingeniería en Computación
Departamento de Tecnologías e Innovación

Trabajo Práctico N° 2 – OPC UA

Docentes:
Ing. Martín Francisco Picó
Ing. Milton Pozzo

Alumnos:
Francisco Bevilacqua; Agustina Peralta

1. Introducción y objetivos

El presente trabajo práctico desarrolla una comunicación industrial basada en OPC UA (Open Platform Communications Unified Architecture) utilizando dos computadoras embebidas Raspberry Pi 4. El sistema implementado adopta una arquitectura cliente-servidor y utiliza un enlace Ethernet punto a punto como medio principal de comunicación entre ambas placas.

La Raspberry Pi 1 cumple el rol de servidor OPC UA y publica un modelo de información propio compuesto por variables de estado y de proceso. La Raspberry Pi 2 funciona como cliente, lee las variables del servidor, escribe sus propios valores y replica los datos recibidos mediante salidas físicas. Cada placa dispone de un pulsador, un LED digital y un LED controlado por PWM.

Además del funcionamiento básico, se implementaron dos mecanismos de acceso a datos: lectura periódica mediante polling y suscripción mediante Subscription/MonitoredItems. También se evaluó el comportamiento frente a una pérdida temporal del enlace Ethernet y se capturó tráfico real con tcpdump para analizarlo posteriormente en Wireshark.

1.1 Objetivo general

Implementar una comunicación cliente-servidor OPC UA sobre microcontroladores o computadoras embebidas, analizando en detalle la estructura de las tramas del protocolo, sus mecanismos de seguridad y los distintos transportes y medios físicos sobre los que puede operar.

1.2 Objetivos específicos

Que los estudiantes puedan:

- Diferenciar los elementos que intervienen en una comunicación OPC UA: servidor, cliente, endpoint, canal seguro, sesión, espacio de direcciones y nodos.
- Diseñar un modelo de información propio (objetos, variables, tipos de datos y NodeIds fijos) y publicarlo en un servidor OPC UA.

- Capturar, decodificar e interpretar campo por campo las tramas del transporte binario OPC UA (UA TCP) con un analizador de protocolos.
- Comparar los mecanismos de acceso a datos por lectura cíclica (Read) y por suscripción (Subscription / MonitoredItems).

  • Configurar y evaluar los distintos modos y políticas de seguridad del protocolo.

  • Identificar los transportes, codificaciones y medios físicos admitidos por OPC UA y contrastarlos con la capa física RS485 empleada en la Tarea Nº1 (MODBus RTU).

2. Arquitectura general y montaje físico

La solución está compuesta por dos Raspberry Pi 4 conectadas directamente mediante un cable Ethernet. La Raspberry Pi 1 actúa como servidor OPC UA y utiliza la dirección 192.168.10.1/24. La Raspberry Pi 2 actúa como cliente y utiliza 192.168.10.2/24. El puerto estándar utilizado por el endpoint del servidor es TCP 4840.



Figura 1. Arquitectura general del prototipo implementado.

2.1 Conexionado de periféricos

Las dos placas utilizan el mismo conexionado. El pulsador se conecta a GPIO17 y se configura con pull-down interno. El LED digital se controla desde GPIO27 y el LED PWM desde GPIO18. Cada LED posee una resistencia serie de 220 Ω.

Elemento	GPIO BCM	Pin físico	Función
Pulsador	GPIO17	11	Entrada digital, pull-down interno
LED PWM	GPIO18	12	Salida PWM para brillo
LED digital	GPIO27	13	Salida digital
Alimentación pulsador	3.3 V	1	Nivel lógico alto
Masa	GND	GND libre	Referencia común

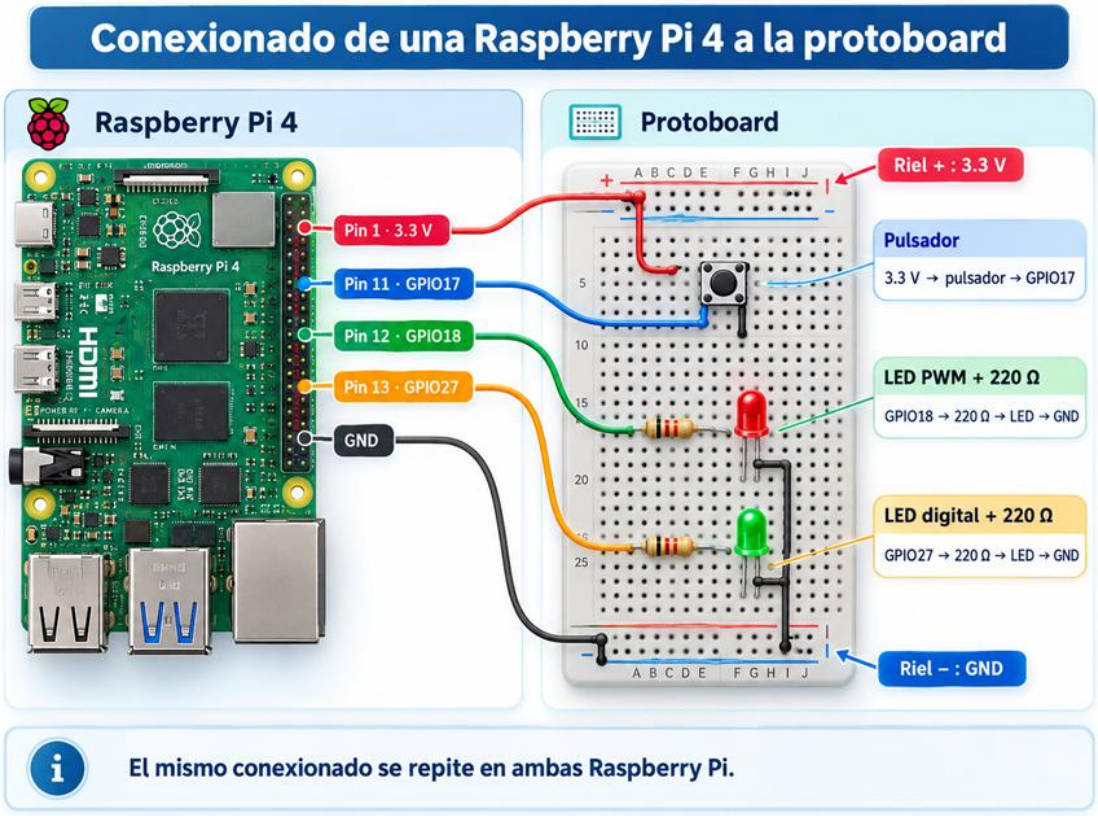


Figura 2. Esquema ilustrativo del conexionado utilizado en cada Raspberry Pi.

3. Parte 1 – Red Ethernet punto a punto

3.1 Configuración de direccionamiento

Como el enlace entre placas no dispone de servidor DHCP, se configuraron direcciones IPv4 estáticas sobre la interfaz eth0. En ambas Raspberry Pi, NetworkManager identificó la conexión Ethernet como `netplan-eth0`.

Equipo	Rol	Dirección IPv4	Máscara
Raspberry Pi 1	Servidor OPC UA	192.168.10.1	/24
Raspberry Pi 2	Cliente OPC UA	192.168.10.2	/24

Configuración aplicada en el servidor:

*sudo nmcli connection modify netplan-eth0 ipv4.method manual *
ipv4.addresses 192.168.10.1/24 ipv4.gateway "" ipv4.dns ""
sudo nmcli connection up netplan-eth0

Configuración aplicada en el cliente:

*sudo nmcli connection modify netplan-eth0 ipv4.method manual *
ipv4.addresses 192.168.10.2/24 ipv4.gateway "" ipv4.dns ""
sudo nmcli connection up netplan-eth0

3.1.1 Verificación del direccionamiento IPv4

Luego de configurar las direcciones IPv4 estáticas sobre la interfaz Ethernet eth0, se verificó la comunicación entre ambas Raspberry Pi utilizando el comando ping. La Raspberry Pi servidor utiliza la dirección 192.168.10.1/24 y la Raspberry Pi cliente 192.168.10.2/24. Ambas direcciones pertenecen a la red 192.168.10.0/24, por lo que los equipos pueden comunicarse directamente sin necesidad de una puerta de enlace o un router intermedio.

Desde el servidor se ejecutó ping -c 4 192.168.10.2 y desde el cliente ping -c 4 192.168.10.1. En ambos sentidos se obtuvo respuesta con 0 % de pérdida de paquetes. Las capturas presentadas en las figuras siguientes constituyen la evidencia de la configuración y conectividad de las direcciones utilizadas. Además, el endpoint opc.tcp://192.168.10.1:4840 emplea la misma red Ethernet sobre eth0 para el intercambio OPC UA entre las dos Raspberry Pi.

3.2 Verificación de conectividad

La conectividad IP se verificó mediante ping en ambos sentidos. Las pruebas realizadas registraron 0 % de pérdida de paquetes y tiempos inferiores al milisegundo.

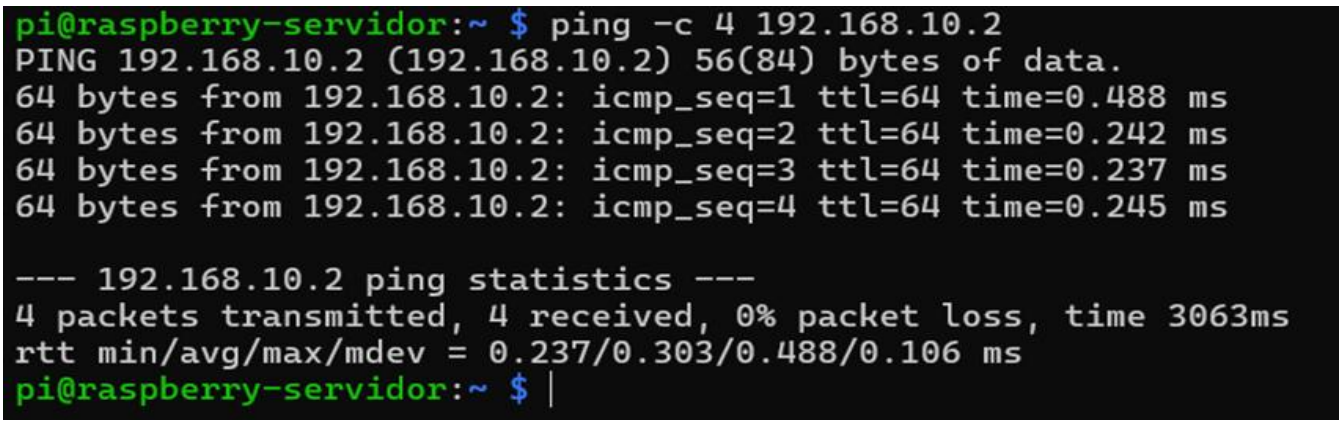


Figura 3. Prueba de conectividad desde el servidor hacia el cliente.

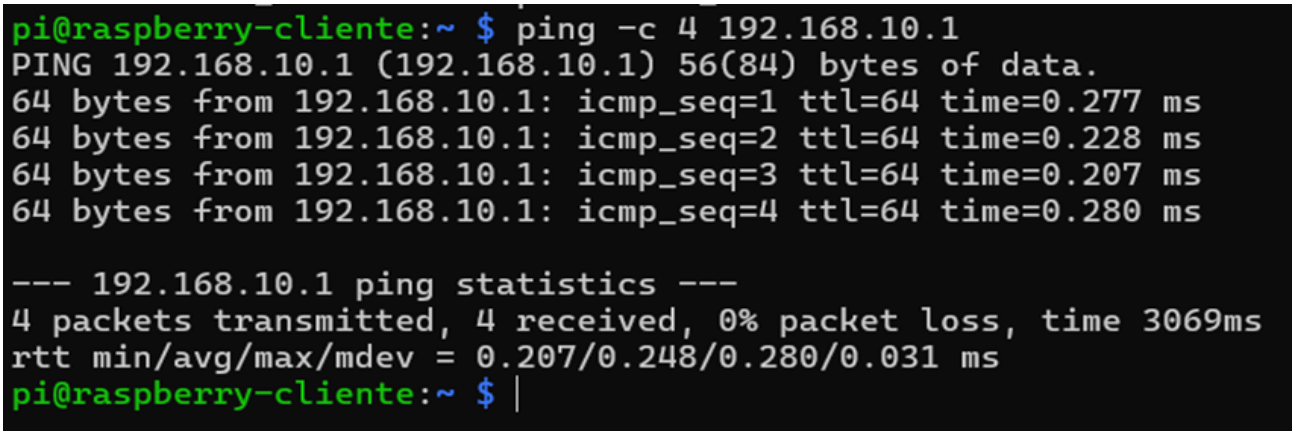


Figura 4. Prueba de conectividad desde el cliente hacia el servidor.

3.3 Capa de enlace, ARP y estado físico

Luego de generar tráfico, `ip neigh` permitió comprobar la asociación entre las direcciones IPv4 y las direcciones MAC del equipo remoto. En el servidor se observó a 192.168.10.2 asociado a la MAC dc:a6:32:52:0e:e3, mientras que el cliente observó a 192.168.10.1 asociado a dc:a6:32:a7:45:1f.

El comando `ethtool eth0` confirmó una velocidad negociada de 1000 Mb/s, modo Full Duplex, autonegociación habilitada y enlace detectado. Este resultado demuestra que las interfaces establecieron un enlace Gigabit Ethernet correcto.

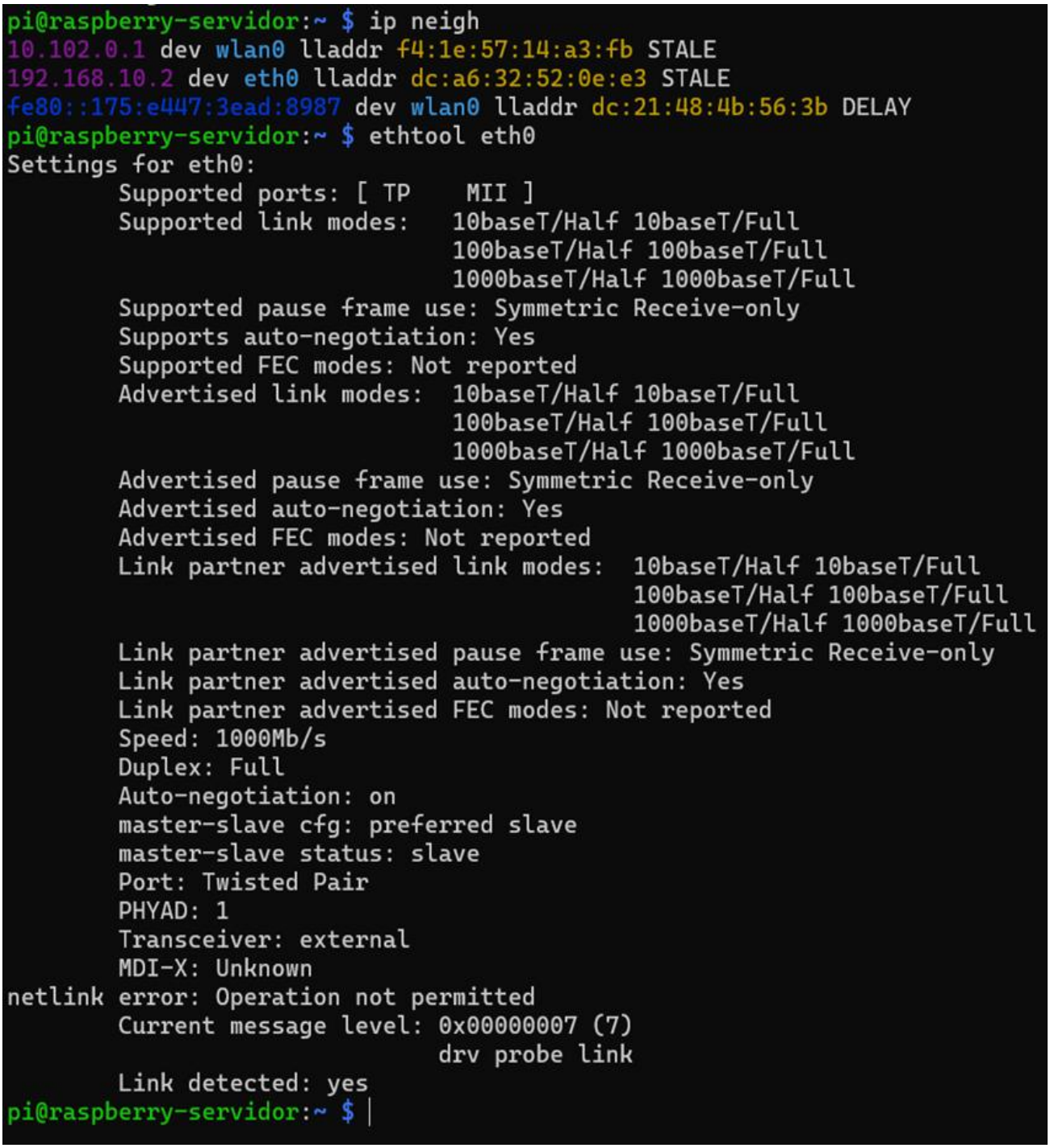


Figura 5. Verificación de vecinos ARP y parámetros del enlace en el servidor.

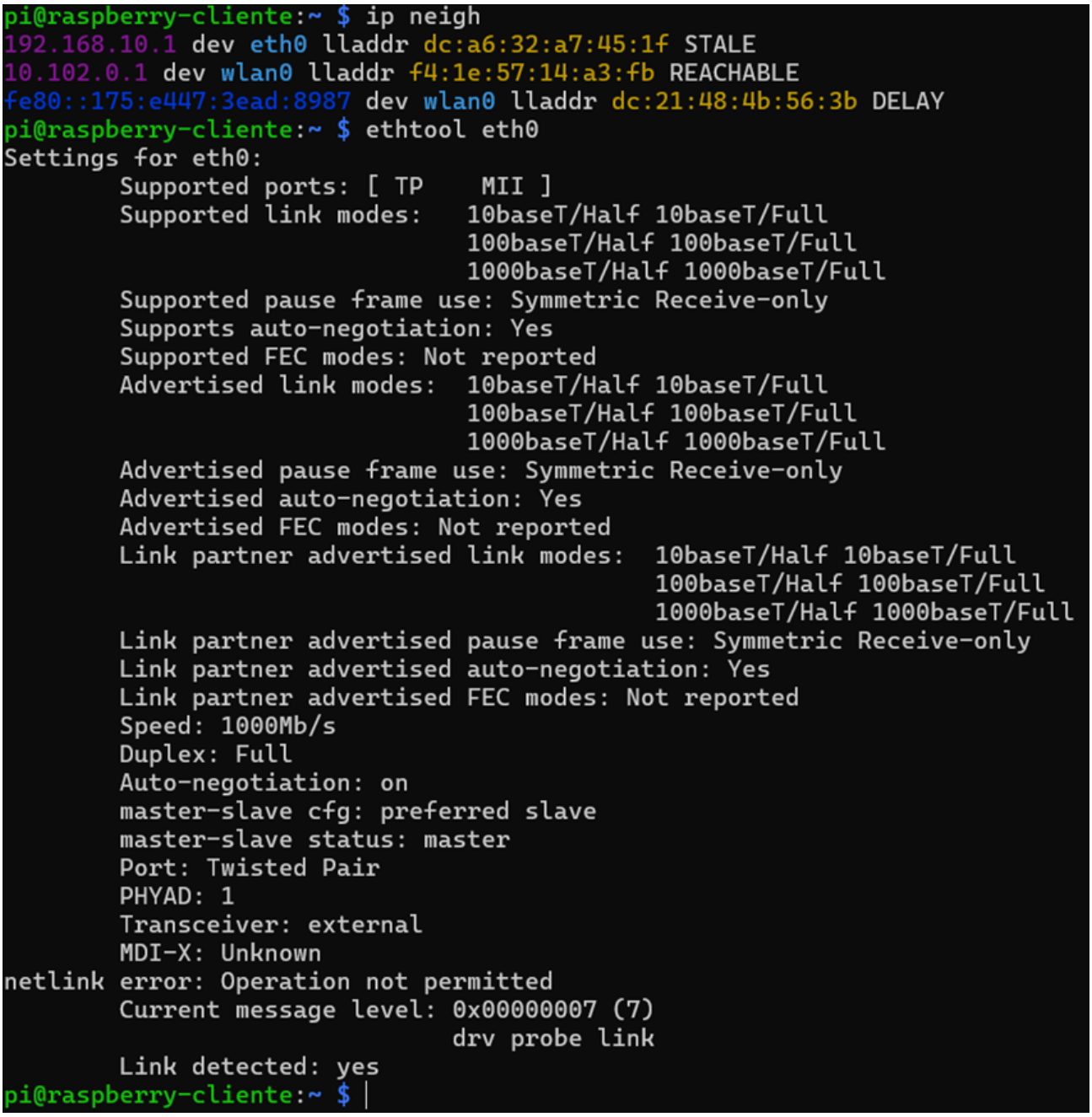


Figura 6. Verificación de vecinos ARP y parámetros del enlace en el cliente.

3.4 Análisis por capas

En la capa física se utilizó un cable Ethernet directo entre las dos interfaces. Las interfaces modernas incorporan Auto MDI-X, por lo que no es necesario seleccionar manualmente un cable cruzado. La autonegociación determina automáticamente la velocidad y el modo dúplex compatibles.

En la capa de enlace, las tramas se entregan empleando direcciones MAC. ARP resuelve la relación entre una dirección IPv4 local y la MAC del host correspondiente. En la capa de red se asignaron direcciones estáticas pertenecientes a la misma subred 192.168.10.0/24; por ello no se requiere una puerta de enlace para el tráfico entre ambas placas.

Si no se configurara una dirección fija y no existiera DHCP, el sistema podría utilizar una dirección de enlace local del rango 169.254.0.0/16. Para este trabajo se evitó esa condición a fin de disponer de endpoints y NodeIds accesibles de manera predecible.

4. Parte 2 – Servidor OPC UA y modelo de información

4.1 Software y librerías

En ambas Raspberry Pi se creó un entorno virtual Python `~/opcua-env`. Se instalaron las librerías `asyncua` para OPC UA, `gpiozero` para acceso a GPIO y `lgpio` como backend GPIO compatible con el sistema operativo utilizado. También fue necesario instalar `swig`, `build-essential`, `python3-dev` y `liblgpio-dev`.

python3 -m venv ~/opcua-env
source ~/opcua-env/bin/activate
pip install asyncua gpiozero
sudo apt install -y swig build-essential python3-dev liblgpio-dev
pip install lgpio

4.2 Endpoint y namespace

El servidor se implementó en Python con asyncua. El endpoint previsto para la comunicación entre Raspberry Pi es `opc.tcp://192.168.10.1:4840`, mientras que el proceso escucha en todas las interfaces. El namespace definido para el trabajo es `urn:unraf:pdci:t2`.

Durante las primeras pruebas el namespace propio apareció como ns=2. Para cumplir los NodeIds fijos exigidos, se corrigió el servidor configurando la URI del trabajo como ApplicationUri y se verificó explícitamente que el índice asignado fuera 1.

```
await server.set_application_uri("urn:unraf:pdci:t2")
idx = await server.get_namespace_index("urn:unraf:pdci:t2")
if idx != 1:
    raise RuntimeError("Se esperaba ns=1")
```

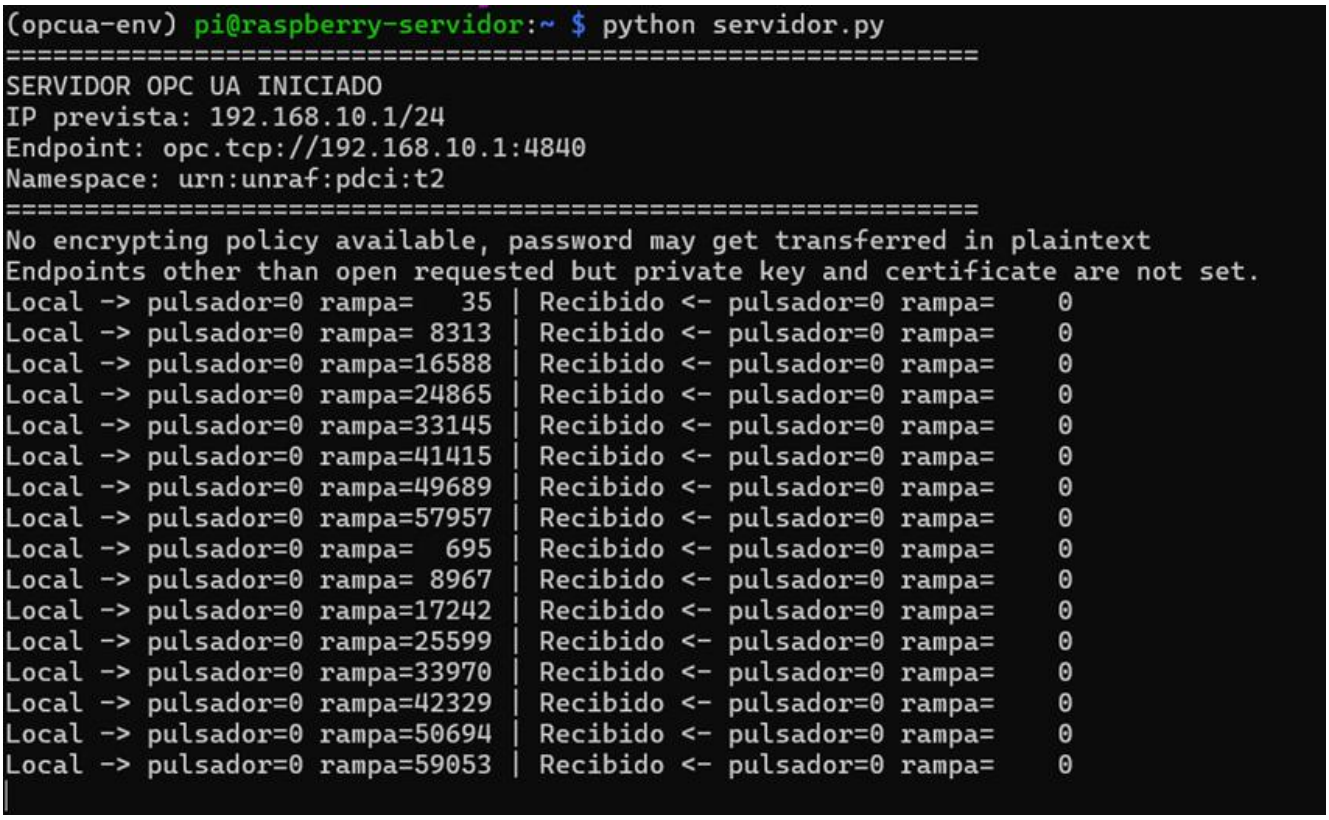


Figura 7. Servidor OPC UA iniciado y generación de rampa diente de sierra.

4.3 Modelo de información

Objeto	Variable	NodeId	Tipo	Acceso
Servidor	EstadoPulsador	ns=1;s=Servidor.EstadoPulsador	Boolean	Lectura
Servidor	Rampa	ns=1;s=Servidor.Rampa	UInt16	Lectura
Cliente	EstadoPulsador	ns=1;s=Cliente.EstadoPulsador	Boolean	Lectura/Escritura
Cliente	Rampa	ns=1;s=Cliente.Rampa	UInt16	Lectura/Escritura

Las variables `Cliente.*` se configuraron como escribibles, mientras que las variables `Servidor.*` se mantienen para lectura remota. Las variables de rampa utilizan valores UInt16 en el rango funcional 0–65535, rango que se verificó durante la generación de las señales, su visualización en UaExpert y el control de los LEDs PWM.



Figura 8. Espacio de direcciones OPC UA implementado.

4.4 Verificación con UaExpert

UaExpert se utilizó desde la notebook como cliente genérico. Para las primeras pruebas se emplearon Security Policy None, Message Security Mode None y autenticación Anonymous. La conexión desde la PC se realizó usando la dirección Wi-Fi del servidor, sin modificar el enlace Ethernet utilizado entre las Raspberry Pi.

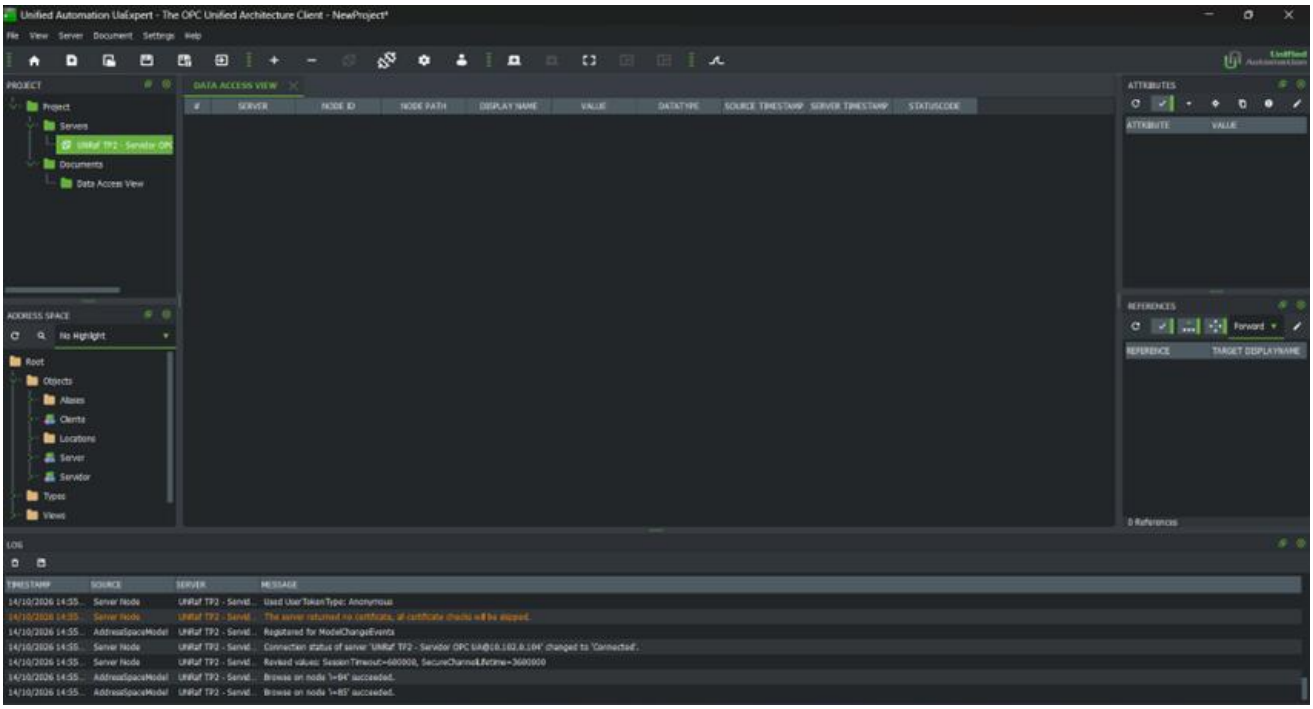


Figura 9. UaExpert conectado al servidor y navegación del Address Space.

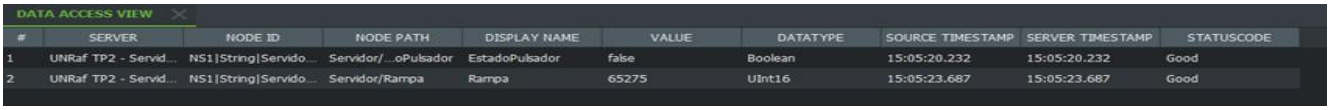


Figura 10. Verificación de variables del servidor en namespace 1.

En UaExpert se comprobaron NodeId, tipo de dato, valor, StatusCode y timestamps de las variables visualizadas. El resultado permitió validar que el modelo publicado coincidía con los NodeIds definidos por la consigna. En una prueba posterior también se verificaron la tendencia de Servidor.Rampa y la escritura remota sobre las variables Cliente.*.

OBSERVACIÓN: La escritura sobre una variable de solo lectura fue probada y UaExpert informó acceso denegado. No se dispone de una captura de pantalla de esa prueba.

4.4.1 Verificación de escritura remota desde UaExpert

Para ampliar la validación del servidor OPC UA, se apagó la Raspberry Pi cliente y se utilizó UaExpert desde la notebook como cliente OPC UA remoto. De esta forma se comprobó que el servidor puede ser operado por cualquier cliente OPC UA compatible que se conecte al endpoint y disponga de permisos de escritura sobre los nodos correspondientes.

En primer lugar se visualizó la variable Servidor.Rampa en Trend View. La señal observada presenta la forma de diente de sierra esperada, confirmando que la rampa continúa siendo generada localmente por la Raspberry servidor aun cuando la Raspberry cliente se encuentra apagada. También se visualizó EstadoPulsador, observándose los cambios entre los estados booleanos 0 y 1.



Figura 11. Vista de tendencia de Servidor.Rampa observada desde UaExpert. Se aprecia la forma diente de sierra generada por la Raspberry servidor.

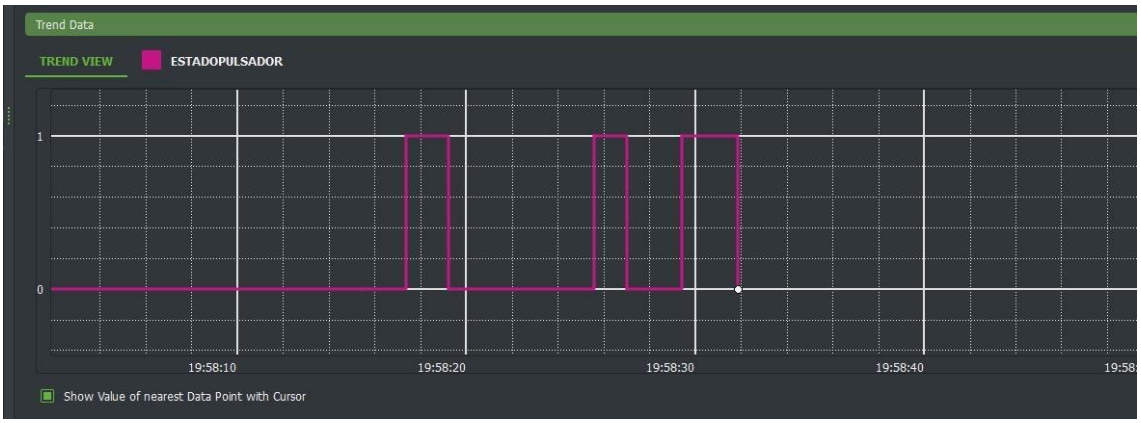


Figura 12. Vista de tendencia de EstadoPulsador en UaExpert, mostrando los cambios entre los estados lógicos 0 y 1.

A continuación se utilizaron las variables escribibles Cliente.EstadoPulsador y Cliente.Rampa directamente desde UaExpert. Al modificar Cliente.EstadoPulsador se verificó el cambio del LED digital conectado al servidor, mientras que al modificar Cliente.Rampa se observó la variación del brillo del LED PWM. En una de las pruebas se escribió manualmente el valor 35000 en Cliente.Rampa, mientras Servidor.Rampa mantuvo su propio valor generado de manera independiente.

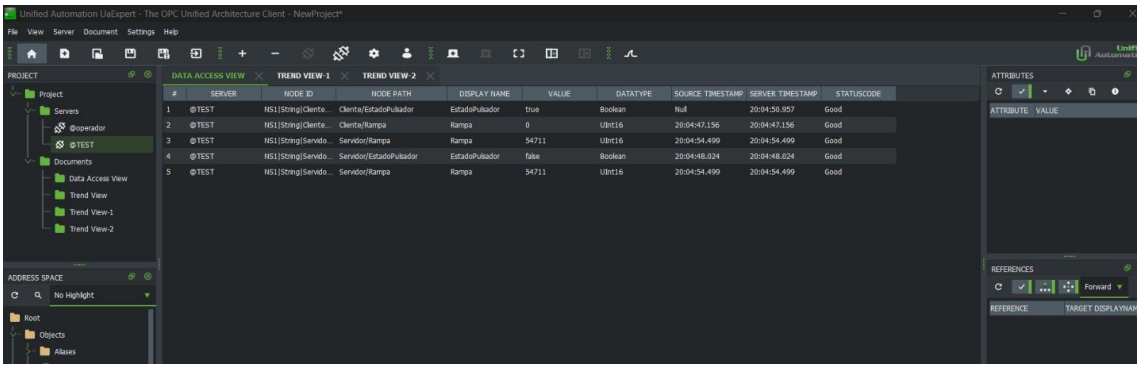


Figura 13. Data Access View de UaExpert con variables Cliente.* y Servidor.*, tipos de datos, timestamps y StatusCode Good.

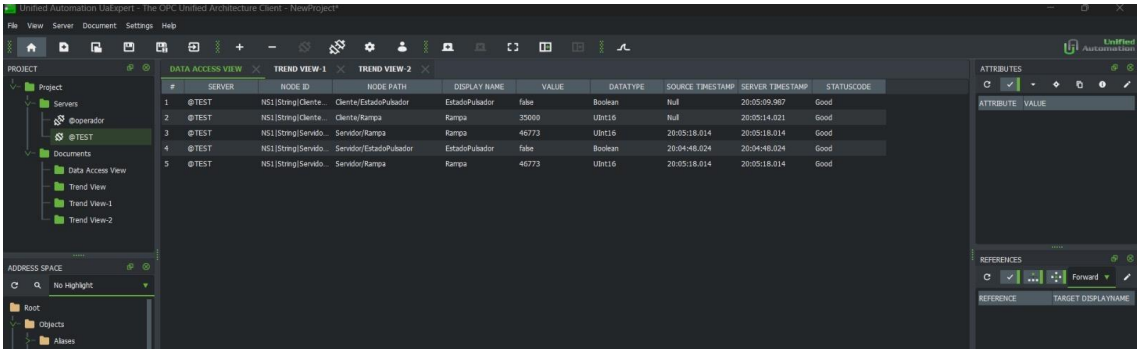


Figura 14. Escritura remota desde UaExpert sobre Cliente.Rampa y Cliente.EstadoPulsador con la Raspberry cliente apagada.

Esta prueba demuestra la interoperabilidad propia de OPC UA: las variables del servidor no dependen de una aplicación cliente específica. La Raspberry Pi cliente puede ser reemplazada temporalmente por UaExpert u otro cliente OPC UA compatible, siempre que éste conozca el endpoint y posea los permisos necesarios para acceder a los nodos. Por lo tanto, el control de las salidas físicas del servidor depende de los valores almacenados en Cliente.*, independientemente del cliente que realice la escritura.

También se realizó una prueba de escritura sobre una variable configurada como de solo lectura. UaExpert rechazó la operación e informó acceso denegado, confirmando que el servidor aplica los permisos definidos para los nodos y no permite modificar variables que fueron publicadas únicamente para lectura. Esta prueba fue observada durante la ejecución, aunque no se conservó una captura de pantalla del mensaje devuelto.

4.5 Diferencia conceptual respecto de MODBus

La diferencia conceptual principal entre MODBus y OPC UA se encuentra en la forma en que ambos protocolos representan y exponen la información del proceso.

En MODBus, la comunicación se basa en un modelo de datos simple organizado en direcciones de memoria o registros. El dispositivo maestro necesita conocer previamente qué dirección corresponde a cada variable, qué tipo de dato representa y cómo debe interpretarse su contenido. Por ejemplo, una determinada magnitud puede encontrarse almacenada en un holding register específico, pero esa dirección por sí sola no aporta información sobre el significado físico del dato, sus unidades, su rango o su función dentro del proceso. Por este motivo, normalmente es necesario disponer de una tabla externa de registros o mapa de memoria proporcionado por el fabricante o definido durante la implementación.

OPC UA utiliza un enfoque diferente. En lugar de exponer solamente posiciones de memoria, el servidor publica un Address Space o espacio de direcciones estructurado y navegable. Dentro de este espacio pueden existir objetos, variables, métodos, tipos de datos, propiedades, atributos y referencias entre nodos.

Cada elemento del Address Space posee un NodeId que permite identificarlo de manera única dentro del servidor. Además del valor de una variable, OPC UA puede proporcionar información adicional asociada al nodo, como su tipo de dato, nombre, nivel de acceso, calidad del valor y marcas de tiempo. De esta manera, el cliente no recibe únicamente un número, sino también información que permite interpretar mejor qué representa ese dato.

Otra diferencia importante es la posibilidad de realizar operaciones de Browse. Un cliente OPC UA puede explorar el Address Space del servidor y descubrir los nodos disponibles, sus relaciones y parte de sus metadatos. Esto reduce la dependencia de una tabla externa fija, ya que el propio servidor describe la estructura de la información que publica.

En el prototipo desarrollado, esta diferencia puede observarse claramente. Las variables no fueron representadas simplemente mediante direcciones numéricas, sino mediante NodeIds descriptivos, por ejemplo:

ns=1;s=Servidor.EstadoPulsador

ns=1;s=Servidor.Rampa

ns=1;s=Cliente.EstadoPulsador

ns=1;s=Cliente.Rampa

Estos identificadores permiten reconocer directamente a qué elemento corresponde cada variable y dentro de qué parte del modelo se encuentra. Además, utilizando UaExpert fue posible navegar el Address Space y observar atributos como el tipo de dato, el valor actual, el StatusCode y los timestamps asociados.

En MODBus, una funcionalidad equivalente requeriría conocer previamente qué registro corresponde a cada una de esas variables y cómo debe interpretarse su contenido. Por ejemplo, el cliente podría leer un registro de 16 bits, pero necesitaría información externa para saber que ese registro corresponde a una rampa, cuál es su rango y cómo debe utilizarse.

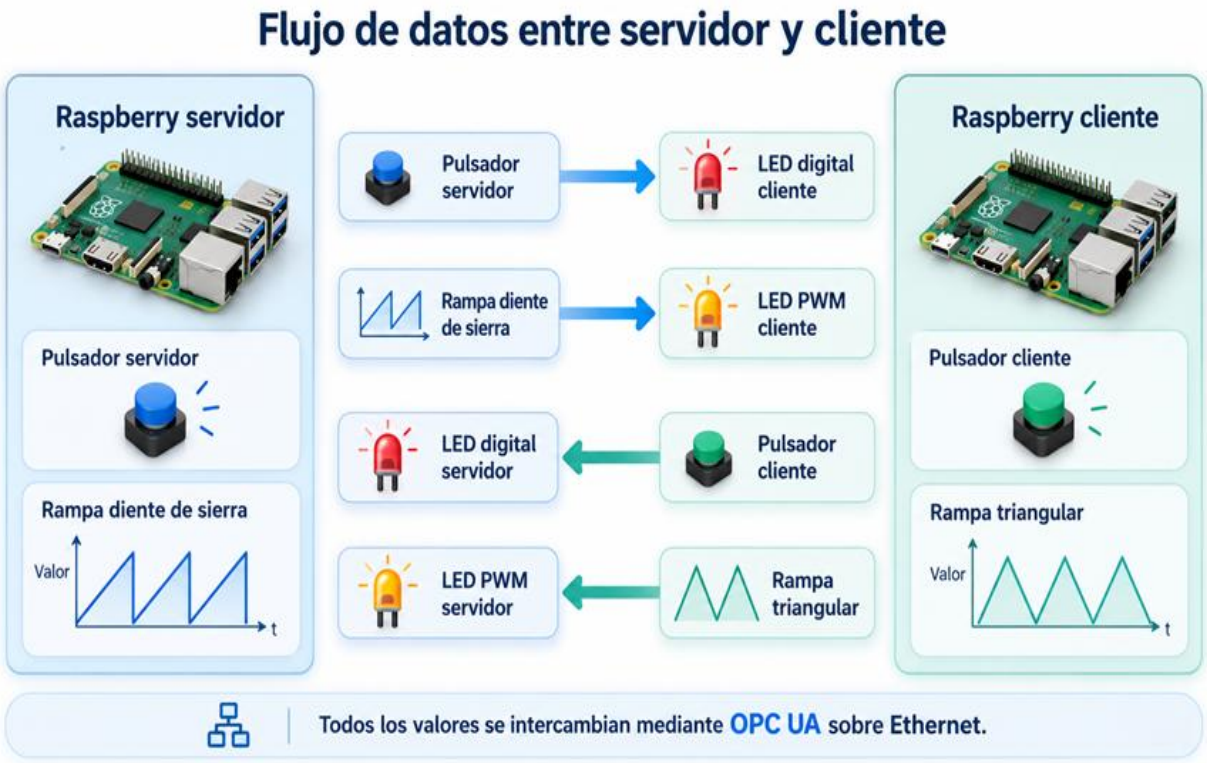
OPC UA también incorpora conceptos adicionales que no forman parte del modelo clásico de MODBus, como sesiones, canales seguros, servicios de lectura y escritura, suscripciones, MonitoredItems y mecanismos de seguridad y autenticación. Esto permite construir sistemas más ricos desde el punto de vista semántico y de interoperabilidad, aunque también genera una mayor complejidad y un mayor overhead de comunicación.

Por lo tanto, MODBus puede considerarse un protocolo más simple y orientado al acceso directo a registros, mientras que OPC UA está orientado a la representación estructurada de la información y a la interoperabilidad entre sistemas. MODBus resulta adecuado cuando se necesita un intercambio sencillo y compacto de datos entre dispositivos, mientras que OPC UA ofrece mayores ventajas cuando se requiere describir el proceso, navegar la información, incorporar metadatos, gestionar sesiones y escalar la comunicación hacia sistemas de supervisión, control o integración industrial.

5. Parte 3 – Cliente OPC UA e intercambio bidireccional

5.1 Generación local y lógica de intercambio

El cliente OPC UA se ejecuta en la Raspberry Pi 2 y se conecta al endpoint del servidor por eth0. Genera una rampa triangular 0–65535 con un período total de 8 s, compuesto por 4 s de subida y 4 s de bajada. El estado del pulsador local se escribe cuando cambia y la rampa local se escribe cada 100 ms.



La lógica del cliente combina procesamiento local con intercambio remoto. En cada ciclo, la Raspberry Pi cliente lee su propio pulsador y genera una rampa triangular de 16 bits. Estos datos locales no controlan directamente sus propias salidas, sino que se escriben en el Address Space del servidor mediante los nodos `Cliente.EstadoPulsador` y `Cliente.Rampa`. De esta manera, el servidor recibe información originada físicamente en la Raspberry cliente.

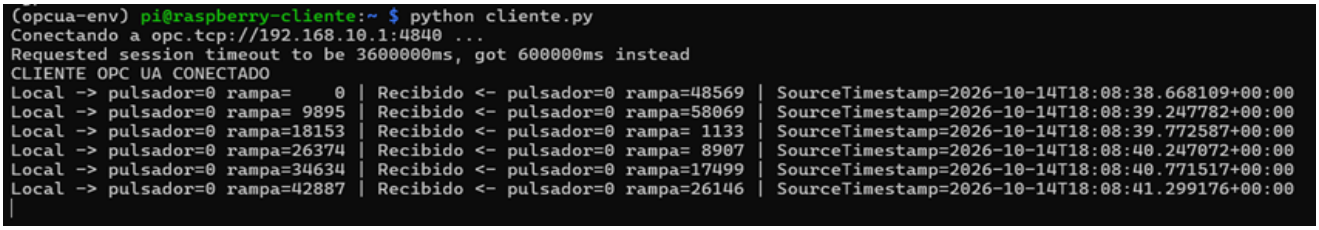
En sentido inverso, el cliente obtiene del servidor los valores de `Servidor.EstadoPulsador` y `Servidor.Rampa`. El estado booleano recibido controla el LED digital del cliente, mientras que el valor UInt16 de la rampa se normaliza al rango de trabajo del PWM y determina el brillo del LED correspondiente. La rampa del servidor tiene forma de diente de sierra, mientras que la del cliente es triangular; esta diferencia facilita reconocer visualmente el origen de cada señal.

El esquema implementado es, por lo tanto, bidireccional: cada Raspberry produce información propia y utiliza información remota para accionar sus salidas. Esto permitió comprobar que el intercambio no se limita a una lectura unilateral, sino que existe transferencia de datos en ambos sentidos a través del mismo servidor OPC UA.

Figura 15. Flujo bidireccional de datos entre servidor y cliente.

5.2 Modo A – Polling

En el modo polling, el cliente realiza servicios Read periódicos sobre `Servidor.EstadoPulsador` y `Servidor.Rampa` cada 100 ms. Los valores recibidos controlan el LED digital y el LED PWM del cliente, respectivamente. El cliente registra además el SourceTimestamp del dato recibido.



En este modo el cliente es quien inicia de forma periódica la obtención de datos. Cada aproximadamente 100 ms ejecuta servicios `Read` sobre los nodos del servidor, recibe los valores y actualiza sus salidas físicas. Esto produce un flujo cíclico de solicitudes y respuestas aun cuando una variable no haya cambiado desde la lectura anterior.

La captura de la consola del cliente permitió verificar simultáneamente el valor local generado, el valor remoto recibido y el `SourceTimestamp` asociado al dato del servidor. El timestamp resulta útil para distinguir el instante asociado a la actualización del valor de la hora local en la que el cliente procesa o muestra la información.

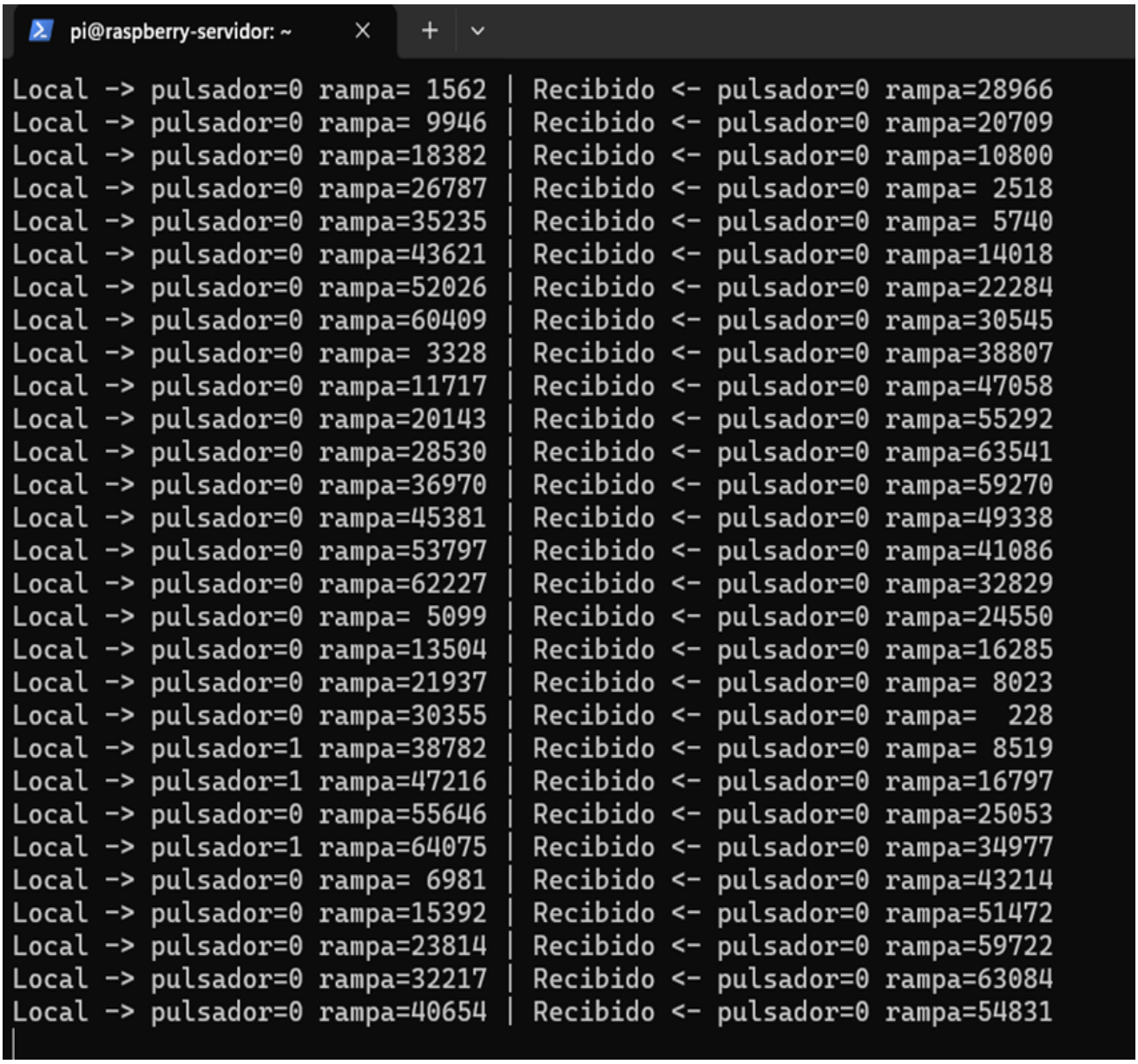
El polling presenta una lógica sencilla y predecible, ya que el cliente decide cuándo leer. Sin embargo, al aumentar la cantidad de variables o disminuir el período de lectura, también

aumenta la cantidad de servicios `Read` transmitidos. Esta característica fue utilizada posteriormente como punto de comparación conceptual frente al mecanismo de suscripción.

Figura 16. Cliente conectado en modo polling con registro de rampa y SourceTimestamp.

5.3 Intercambio bidireccional verificado

Las pruebas físicas confirmaron el intercambio en ambos sentidos: el pulsador del servidor controla el LED digital del cliente y la rampa diente de sierra controla el LED PWM del cliente; el pulsador del cliente controla el LED digital del servidor y la rampa triangular controla el LED PWM del servidor.



Desde el punto de vista del servidor, la consola muestra en paralelo los datos generados localmente y los valores recibidos desde el cliente. Esto permitió verificar que `Cliente.EstadoPulsador` y `Cliente.Rampa` eran actualizados por la Raspberry cliente y que esos valores se utilizaban para gobernar el LED digital y el LED PWM del servidor.

Desde el punto de vista del cliente se observó el comportamiento complementario: la Raspberry continuaba generando su propia rampa triangular y leyendo su pulsador, mientras recibía el estado del pulsador y la rampa diente de sierra del servidor. Los valores remotos eran aplicados a las salidas físicas de la placa cliente.

La utilización de formas de onda diferentes fue una decisión funcional para facilitar la validación. Si ambas rampas hubieran tenido la misma forma, habría sido más difícil distinguir visualmente si un LED PWM respondía al generador local o a la variable recibida desde la otra Raspberry.

Figura 17. Registro del servidor durante el intercambio bidireccional.

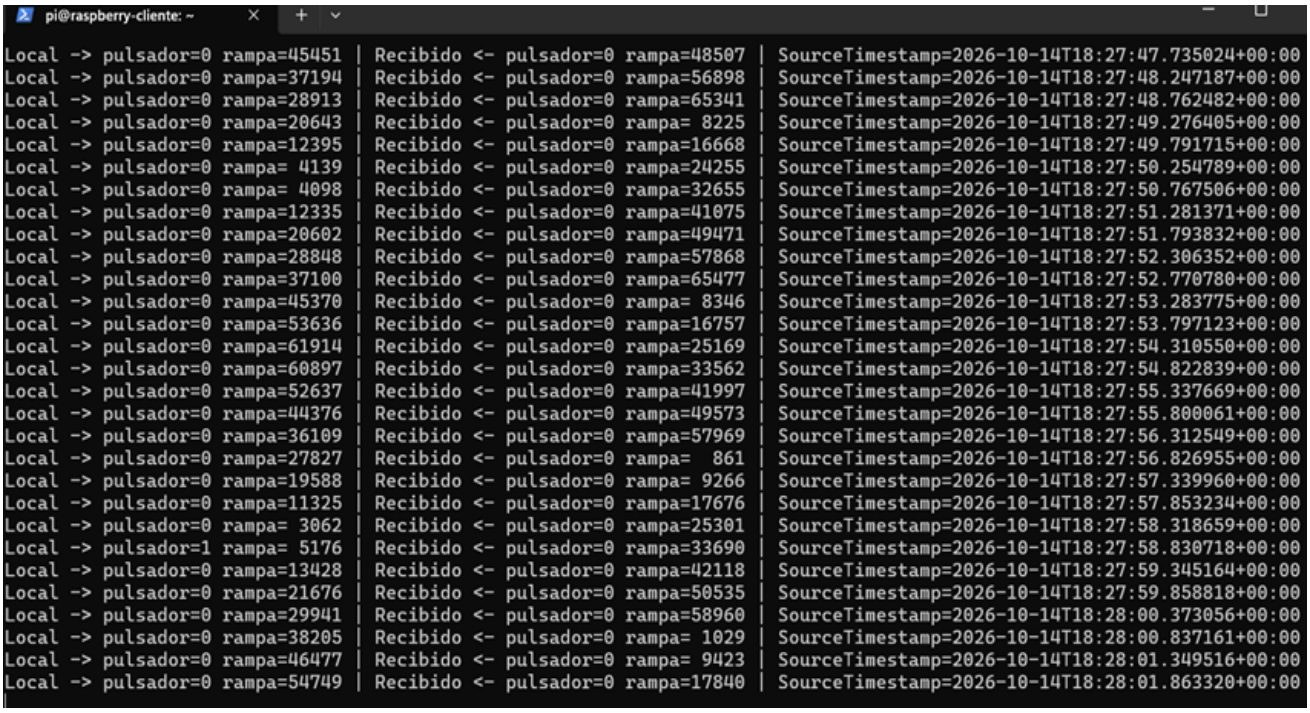


Figura 18. Registro del cliente durante el intercambio bidireccional.

6. Lectura cíclica y suscripciones

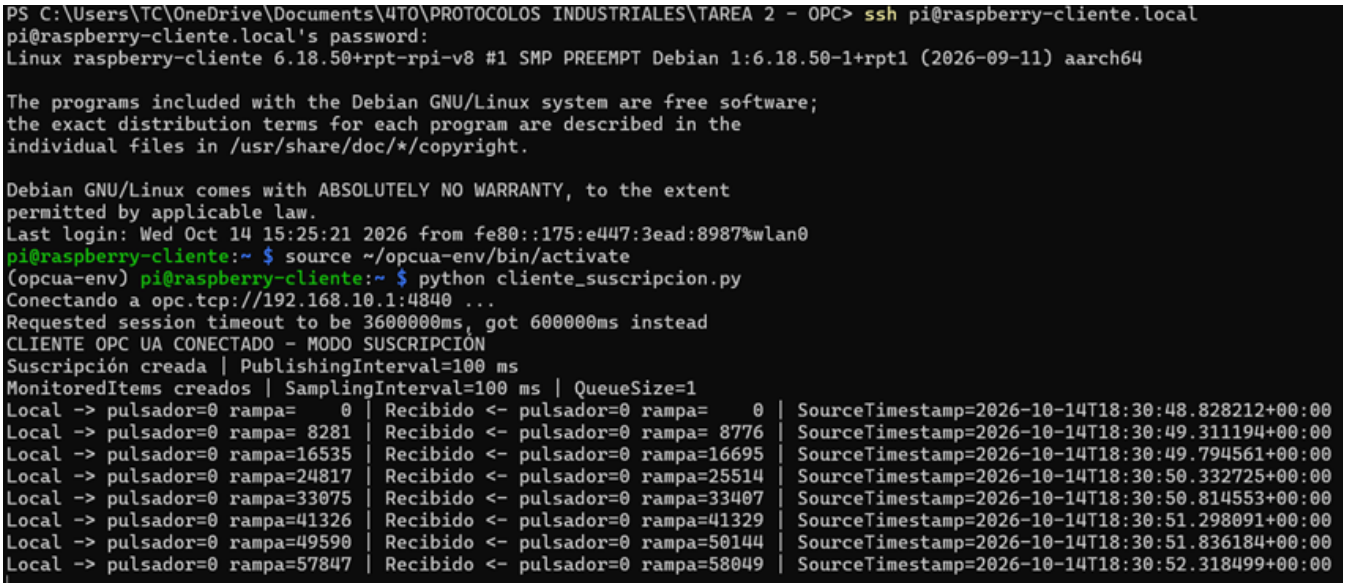
6.1 Modo B – Subscription / MonitoredItems

El segundo modo utiliza una suscripción OPC UA. Se configuró un PublishingInterval de 100 ms y se crearon MonitoredItems para `Servidor.EstadoPulsador` y `Servidor.Rampa` con SamplingInterval de 100 ms y QueueSize igual a 1.

Parámetro	**Valor**
PublishingInterval	100 ms

SamplingInterval	100 ms
QueueSize	1
Nodos monitoreados	Servidor.EstadoPulsador / Servidor.Rampa

En una primera versión ambos MonitoredItems utilizaron un ClientHandle incorrecto, lo que provocó que la rampa se notificara correctamente pero el estado del pulsador del servidor no se reflejara en el cliente. La versión `cliente_suscripcion_v2.py` utiliza la API de alto nivel `subscribe_data_change`, permitiendo que asyncua asigne handles distintos.



En el modo de suscripción el cliente deja de realizar lecturas cíclicas independientes sobre cada variable y solicita al servidor que supervise los nodos de interés. La `Subscription` se creó con un `PublishingInterval` de 100 ms y sobre ella se definieron dos `MonitoredItems`, uno para el estado del pulsador y otro para la rampa del servidor.

El `SamplingInterval` de 100 ms establece la frecuencia con la que el servidor evalúa cada valor monitoreado. `QueueSize = 1` indica que, si se producen actualizaciones más rápido de lo que pueden entregarse, se conserva únicamente el valor más reciente en la cola de ese MonitoredItem. Para este prototipo esta configuración es coherente con el objetivo de representar el estado actual de las señales físicas sin acumular muestras antiguas.

La primera implementación presentó un problema práctico: ambos elementos monitoreados habían quedado asociados a identificadores de cliente incorrectos y el cambio del pulsador no se reflejaba correctamente, aunque la rampa sí llegaba. La corrección consistió en

utilizar `subscribe_data_change()` para cada nodo, permitiendo que `asyncua` gestione identificadores distintos. La Figura 20 documenta la versión corregida, donde ya se recibe tanto la rampa como el estado del pulsador.

Figura 19. Primera ejecución del cliente en modo suscripción.

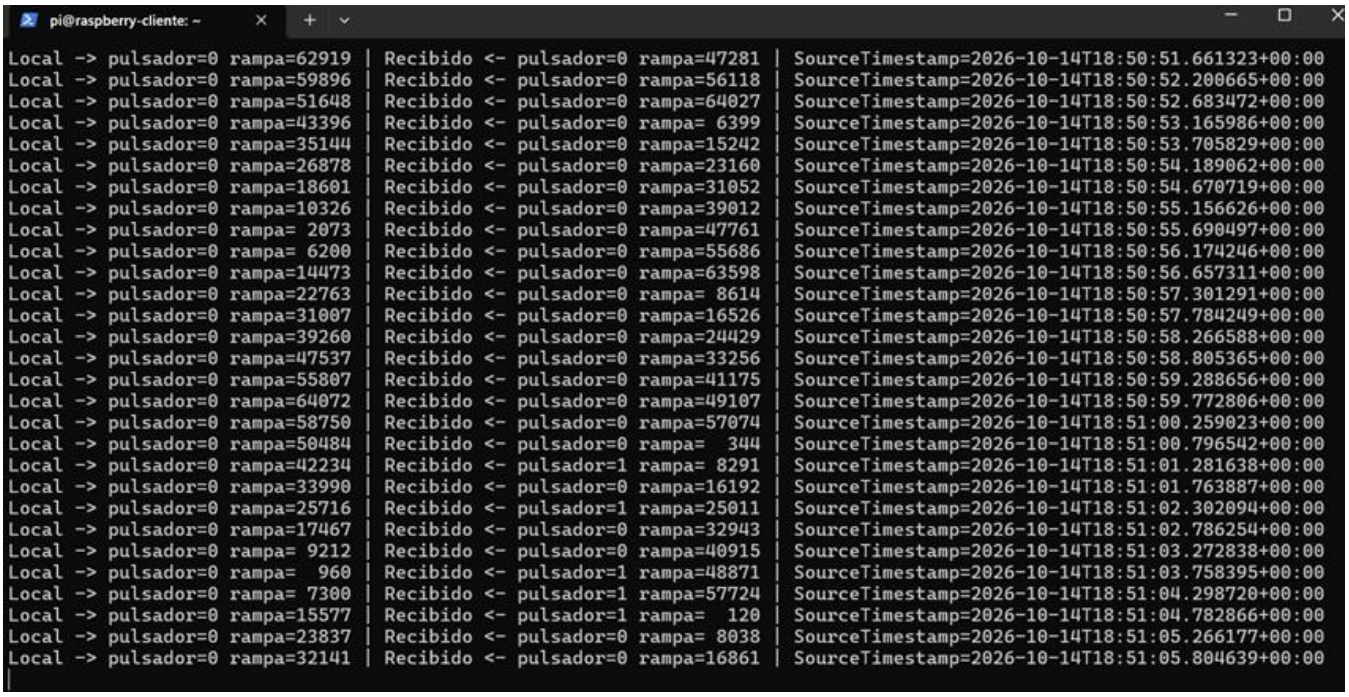


Figura 20. Versión corregida: recepción del pulsador mediante MonitoredItems.

6.2 Mecanismo Publish

En OPC UA, el mecanismo de Subscription / MonitoredItems permite que un cliente supervise variables del servidor sin necesidad de ejecutar continuamente servicios Read sobre cada una de ellas.

Una Subscription representa el vínculo lógico mediante el cual el cliente solicita al servidor que supervise determinados datos y le informe cuando se produzcan cambios. Dentro de cada suscripción se crean uno o más MonitoredItems, cada uno asociado a un nodo o atributo específico del Address Space.

En el prototipo desarrollado, la Raspberry Pi cliente crea MonitoredItems para las variables:

ns=1;s=Servidor.EstadoPulsador

ns=1;s=Servidor.Rampa

Estas variables son supervisadas con un SamplingInterval de 100 ms. Esto significa que el servidor evalúa periódicamente el valor de cada variable monitoreada para detectar cambios.

La suscripción, a su vez, utiliza un PublishingInterval de 100 ms. Este parámetro determina el intervalo con el que el servidor evalúa si debe enviar información de la suscripción hacia el cliente.

La entrega de las notificaciones se realiza mediante el mecanismo Publish. El cliente envía solicitudes PublishRequest al servidor y mantiene solicitudes pendientes disponibles. Cuando llega el momento de publicar y existen cambios en alguno de los MonitoredItems, el servidor utiliza una de esas solicitudes para responder mediante un PublishResponse.

El PublishResponse puede contener una o varias notificaciones de cambio de datos. En este caso, cuando cambia Servidor.EstadoPulsador o Servidor.Rampa, el servidor incluye el nuevo valor correspondiente en la respuesta. El cliente recibe la notificación y actualiza sus salidas físicas: el estado del pulsador remoto controla el LED digital y el valor de la rampa remota determina el brillo del LED PWM.

Este comportamiento es diferente al modo polling. En polling, el cliente debe iniciar explícitamente un servicio Read cada vez que desea conocer el valor actualizado de una variable. Por ejemplo, si el ciclo de lectura es de 100 ms, el cliente envía solicitudes de lectura continuamente aunque el dato no haya cambiado.

Con una suscripción, en cambio, el servidor mantiene internamente el seguimiento de los MonitoredItems y genera notificaciones cuando corresponde. Por este motivo, el cliente no necesita realizar un Read independiente sobre cada nodo en cada ciclo.

Otro aspecto importante es que el servidor puede responder a un PublishRequest incluso cuando no existen nuevos cambios de datos. En ese caso puede enviar un mensaje de KeepAlive, cuya función es indicar al cliente que la suscripción continúa activa y que la comunicación con el servidor sigue funcionando.

El intercambio puede resumirse de la siguiente manera:

Cliente → PublishRequest → Servidor

El servidor mantiene la solicitud pendiente hasta que ocurre alguna de las siguientes situaciones:

- existe una notificación de cambio de datos;
- corresponde enviar información de la suscripción;
- se alcanza el momento de enviar un KeepAlive.

Cuando ocurre alguna de estas condiciones:

Servidor → PublishResponse → Cliente

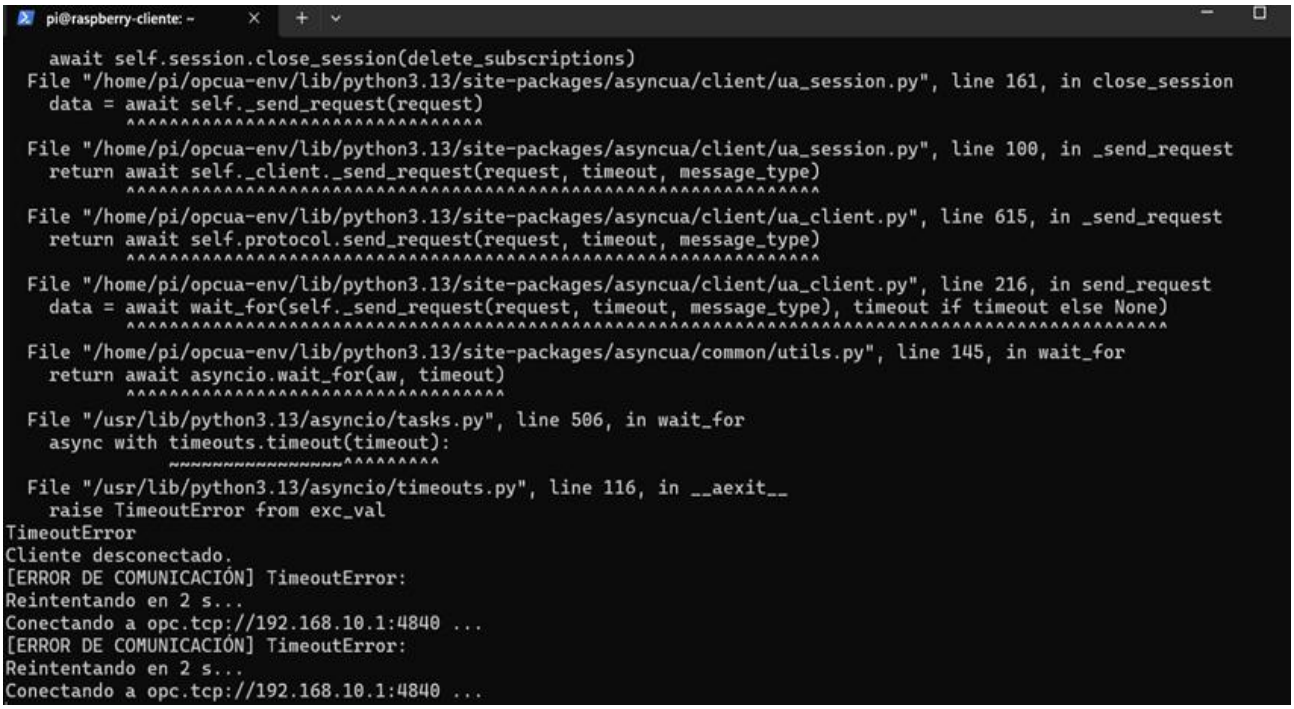
El cliente, después de recibir la respuesta, mantiene nuevas solicitudes PublishRequest disponibles para que el mecanismo pueda continuar funcionando de manera permanente.

En las capturas realizadas con Wireshark se observó claramente este comportamiento, con una sucesión continua de mensajes PublishRequest enviados desde la Raspberry cliente y PublishResponse enviados por la Raspberry servidor. Esto permitió comprobar experimentalmente el funcionamiento del mecanismo de suscripción de OPC UA.

Por lo tanto, el mecanismo Publish permite desacoplar la supervisión de variables de las lecturas periódicas realizadas directamente por el cliente. El servidor asume la tarea de monitorear los datos y notificar los cambios, mientras que el cliente mantiene solicitudes Publish disponibles para recibir dichas notificaciones. Esta característica hace que el modelo de suscripción sea especialmente útil cuando se desea supervisar múltiples variables de proceso de manera organizada y basada en eventos de cambio.

7. Prueba de desconexión y reconexión

Con el sistema funcionando se desconectó el cable Ethernet. El cliente detectó la pérdida de comunicación mediante TimeoutError y comenzó a reintentar la conexión aproximadamente cada 2 s. Durante la interrupción, las salidas que dependen de información remota quedan congeladas en su último estado disponible.

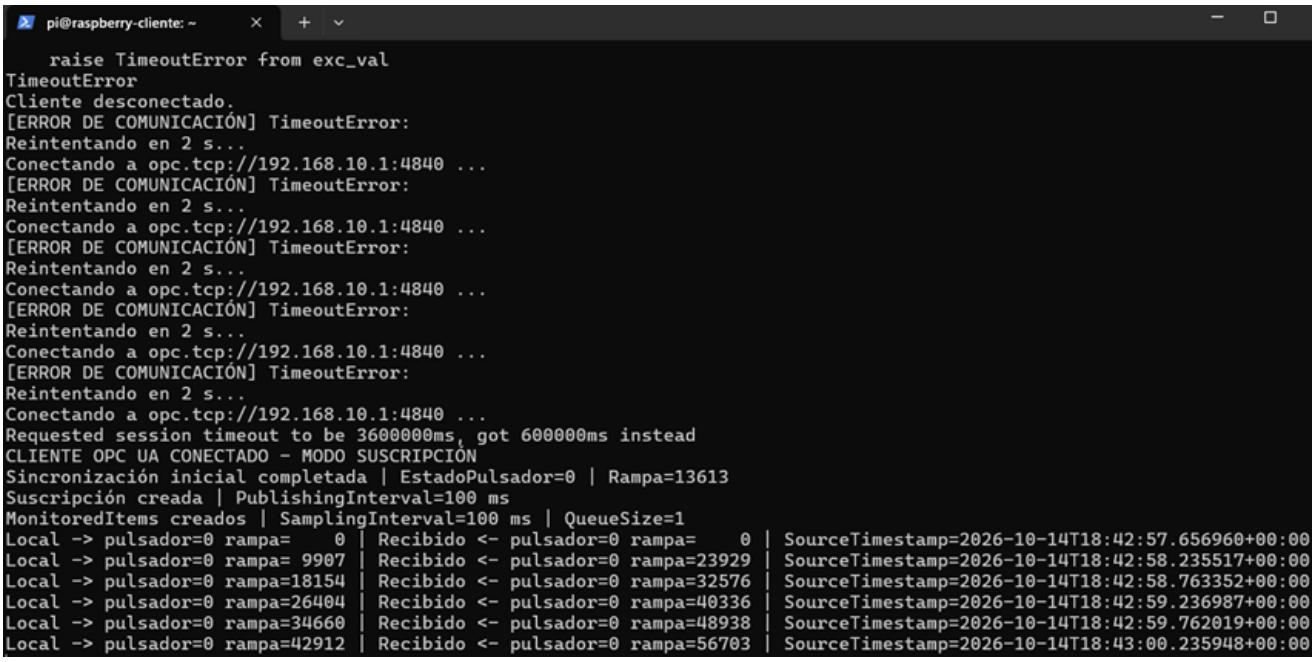


7.1 Comportamiento durante la pérdida del enlace

Durante la prueba se comprobó experimentalmente que, al desconectar físicamente el enlace Ethernet, las variables provenientes de la Raspberry remota dejaron de actualizarse. Como consecuencia, el LED PWM controlado por información remota quedó congelado en el último valor recibido antes de la interrupción. En paralelo, el generador local continuó ejecutándose normalmente, ya que la generación de la rampa local no depende de la comunicación OPC UA.

La prueba permitió diferenciar claramente el procesamiento local de las funciones dependientes del enlace: el generador local continuó funcionando, mientras que las salidas asociadas a datos remotos mantuvieron su último estado disponible. Este comportamiento fue registrado en video y observado durante la demostración práctica del prototipo.

Al restablecer el cable, el cliente vuelve a establecer la comunicación y recrea la suscripción. Para evitar que los LEDs conservaran un valor antiguo si la variable remota no cambiaba de manera inmediata, se incorporó una sincronización inicial: antes de crear la suscripción se leen una vez los valores actuales del servidor y se actualizan las salidas locales.



La interrupción del cable no detiene los generadores locales: cada Raspberry continúa ejecutando su lógica propia. Lo que se pierde es la actualización de las variables remotas. Por este motivo, el LED PWM dependiente de la rampa recibida conserva el último nivel de brillo disponible mientras la rampa local sigue evolucionando. Este comportamiento fue registrado en video y observado durante la demostración práctica.

Cuando el enlace vuelve a estar disponible no alcanza con recuperar únicamente conectividad IP. El cliente debe restablecer la conexión TCP y el contexto OPC UA, obtener nuevamente las referencias a los nodos y recrear la suscripción y sus MonitoredItems. La implementación incorpora reintentos automáticos aproximadamente cada 2 s hasta recuperar la comunicación.

La lectura inicial realizada después de reconectar evita una condición importante: si un valor remoto permaneciera estable, podría no generarse inmediatamente una nueva notificación de cambio. Al leer una vez el estado actual antes de recrear la supervisión continua, los LEDs quedan sincronizados con el valor vigente del servidor desde el comienzo de la nueva sesión.

Figura 22. Reconexión OPC UA con sincronización inicial de valores.

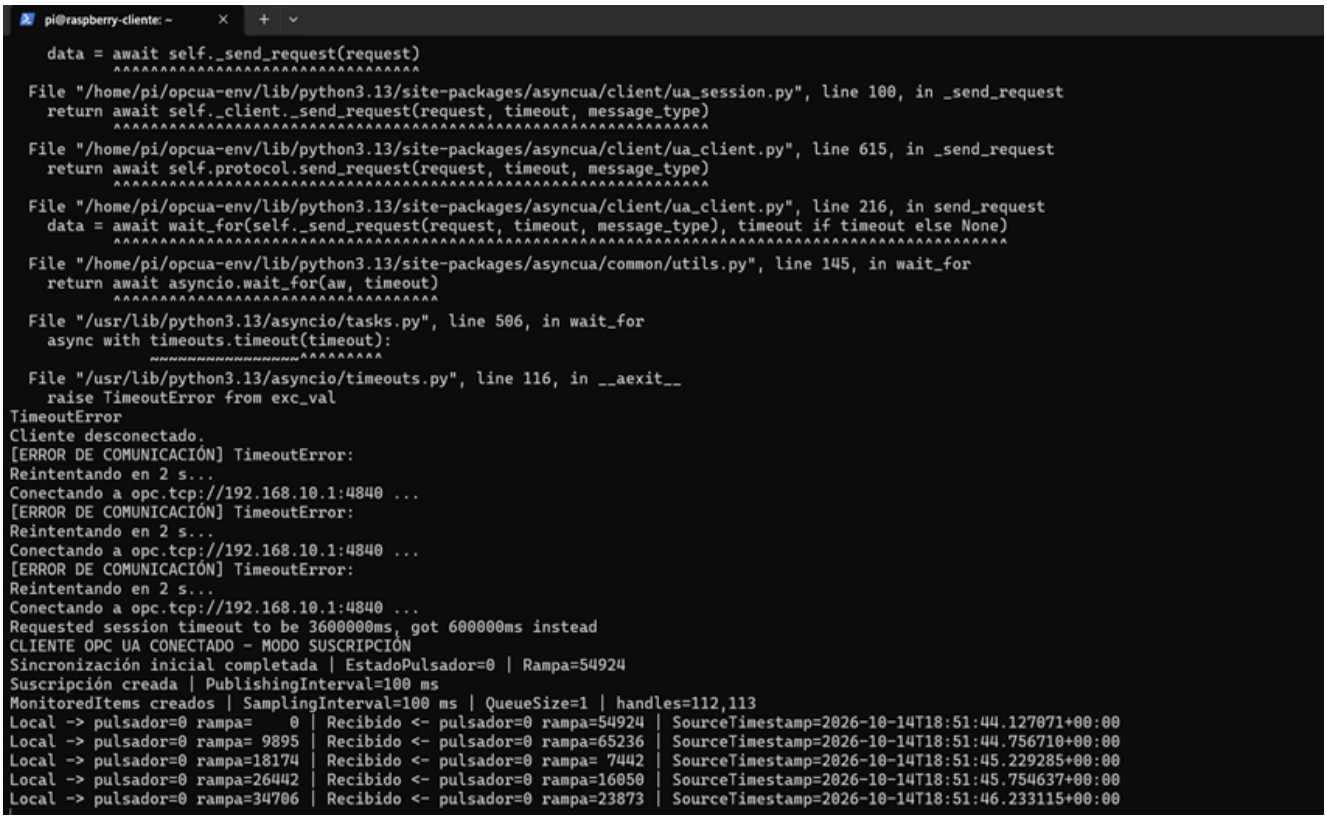


Figura 23. Recuperación completa del cliente y actualización de salidas luego de reconectar.

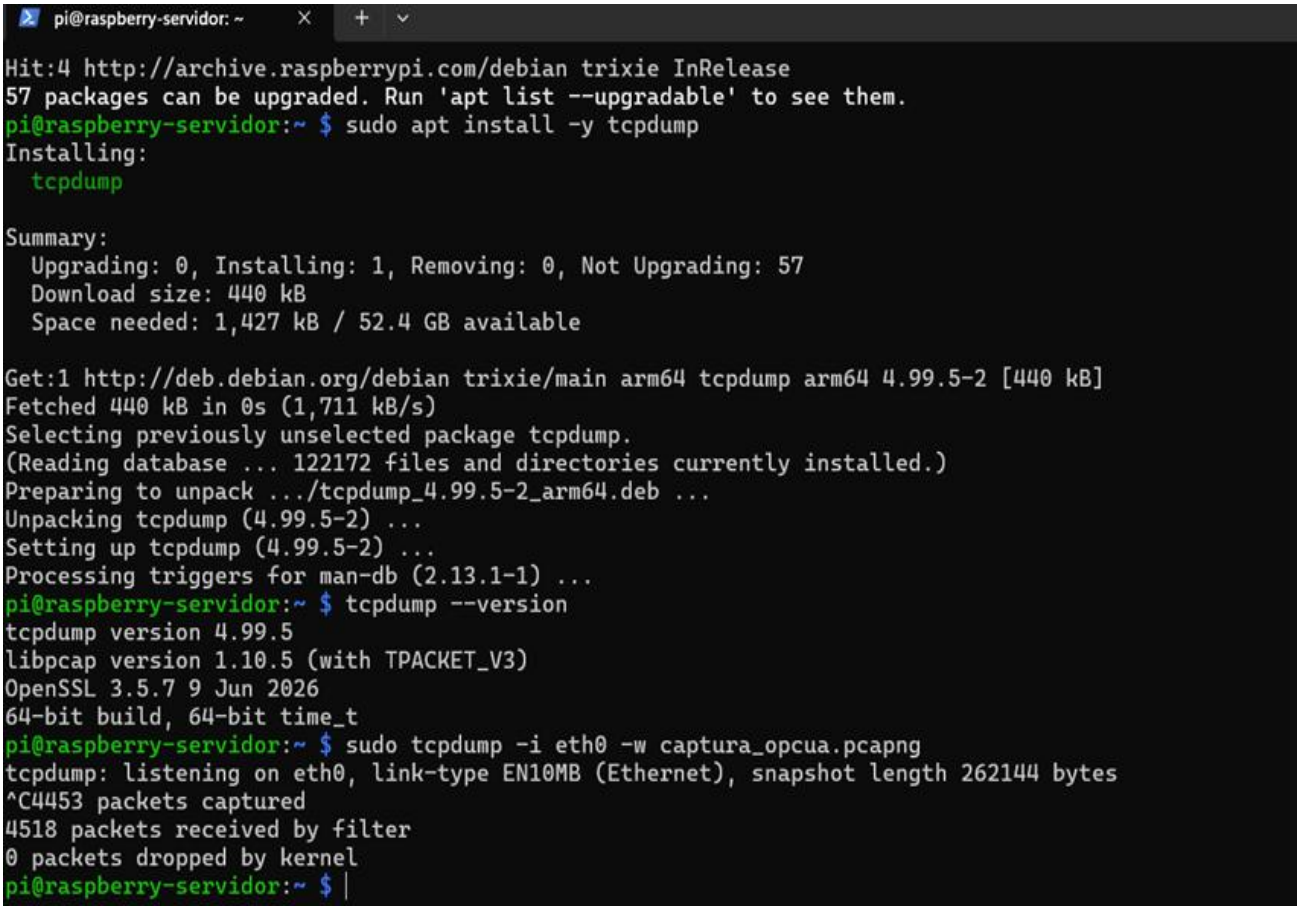
La prueba demuestra que la recuperación no se limita a recuperar la conexión TCP: también es necesario volver a establecer el contexto OPC UA y asegurar que el estado físico se resincronice con los valores vigentes del servidor.

8. Parte 4 – Captura y análisis de tramas OPC UA

8.1 Captura con tcpdump

Debido a que el enlace OPC UA es directo entre las dos Raspberry Pi, la notebook no participa en ese tráfico. Por este motivo la captura se realizó en la interfaz eth0 del servidor mediante `tcpdump` y luego se copió el archivo `.pcapng` a Windows para analizarlo con Wireshark.

sudo tcpdump -i eth0 -w captura_opcua.pcapng



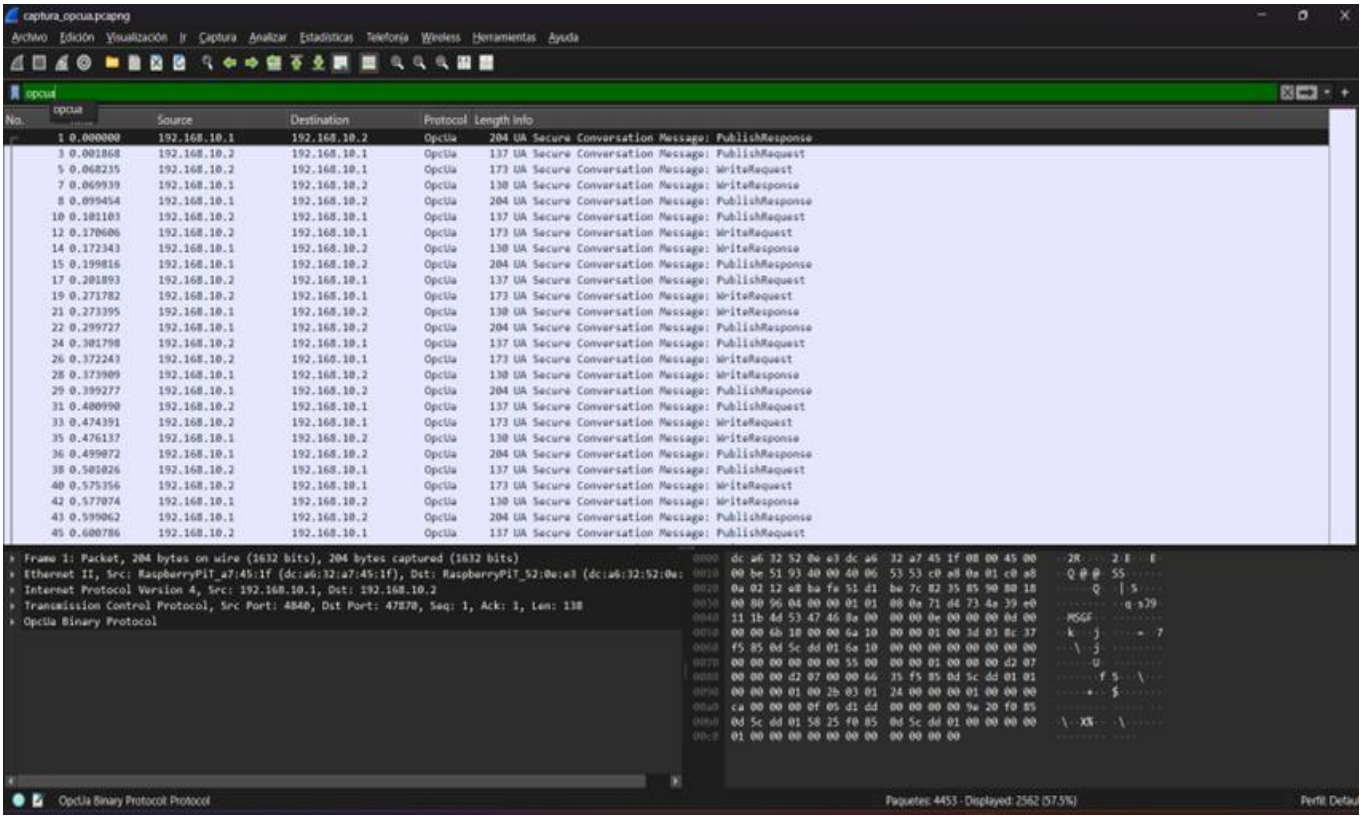
La elección de `tcpdump` sobre `eth0` garantiza que la captura corresponda al enlace punto a punto entre las Raspberry y no al tráfico de administración Wi-Fi utilizado para SSH o UaExpert. El archivo obtenido conserva las tramas reales para analizarlas posteriormente sin modificar el funcionamiento del prototipo durante la prueba.

La captura utilizada contiene 4453 paquetes y cubre aproximadamente 70 s de funcionamiento. No se registraron paquetes descartados por el kernel durante la adquisición, lo cual brinda una base adecuada para el análisis posterior de la secuencia TCP y de los servicios OPC UA.

Figura 24. Captura tcpdump realizada sobre eth0: 4453 paquetes capturados y 0 descartados por el kernel.

8.2 Tráfico observado en Wireshark

El filtro `opcua` permitió aislar los mensajes OPC UA decodificados. En la captura obtenida se observaron numerosos PublishRequest/PublishResponse y WriteRequest/WriteResponse correspondientes al funcionamiento normal del cliente por suscripción.



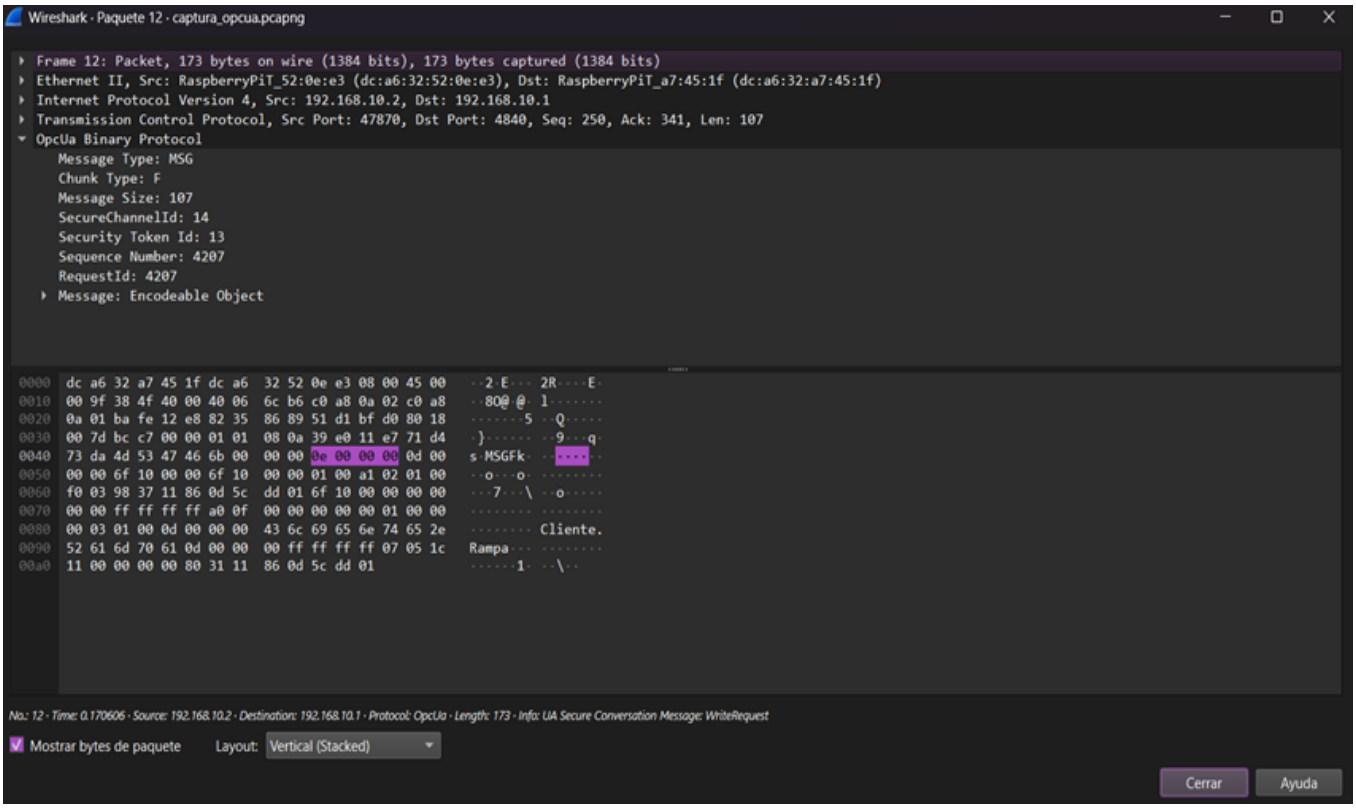
Al aplicar el filtro `opcua` se aislaron 2562 mensajes decodificados como OPC UA. En el tráfico de operación normal predominan los intercambios `PublishRequest/PublishResponse` propios de la suscripción y los `WriteRequest/WriteResponse` utilizados por el cliente para escribir sus variables locales en el servidor.

En el análisis de la captura se contabilizaron 609 `PublishRequest`, 608 `PublishResponse`, 603 `WriteRequest`, 603 `WriteResponse`, 61 `ReadRequest` y 61 `ReadResponse`. La presencia sostenida de mensajes Publish confirma el funcionamiento del modo de suscripción, mientras que las escrituras periódicas corresponden principalmente a la rampa y al estado enviados por la Raspberry cliente.

Figura 25. Tráfico OPC UA filtrado en Wireshark: mensajes Publish y Write.

8.3 WriteRequest sobre Cliente.Rampa

En un WriteRequest analizado se observó un frame de 173 bytes, originado en 192.168.10.2 y destinado a 192.168.10.1. El puerto TCP destino es 4840 y el payload OPC UA posee 107 bytes. El mensaje contiene MessageType MSG, ChunkType F y los campos de canal seguro y secuencia.



El sentido de la trama coincide con la arquitectura implementada: el cliente 192.168.10.2 utiliza un puerto TCP efímero y envía la solicitud al puerto 4840 del servidor 192.168.10.1. Dentro del mensaje OPC UA se identifica el servicio `WriteRequest`, el NodeId de la variable a modificar y el valor codificado que debe almacenar el servidor.

En el caso de `Cliente.Rampa`, el dato de proceso es un `UInt16`, por lo que el valor útil ocupa 2 bytes. El tamaño total del frame es considerablemente mayor debido a las cabeceras de Ethernet, IPv4 y TCP, además de la estructura propia de UA TCP, el canal seguro, la secuencia y la información necesaria para identificar el servicio y el nodo.

Figura 26. Detalle de un WriteRequest dirigido desde el cliente al servidor.

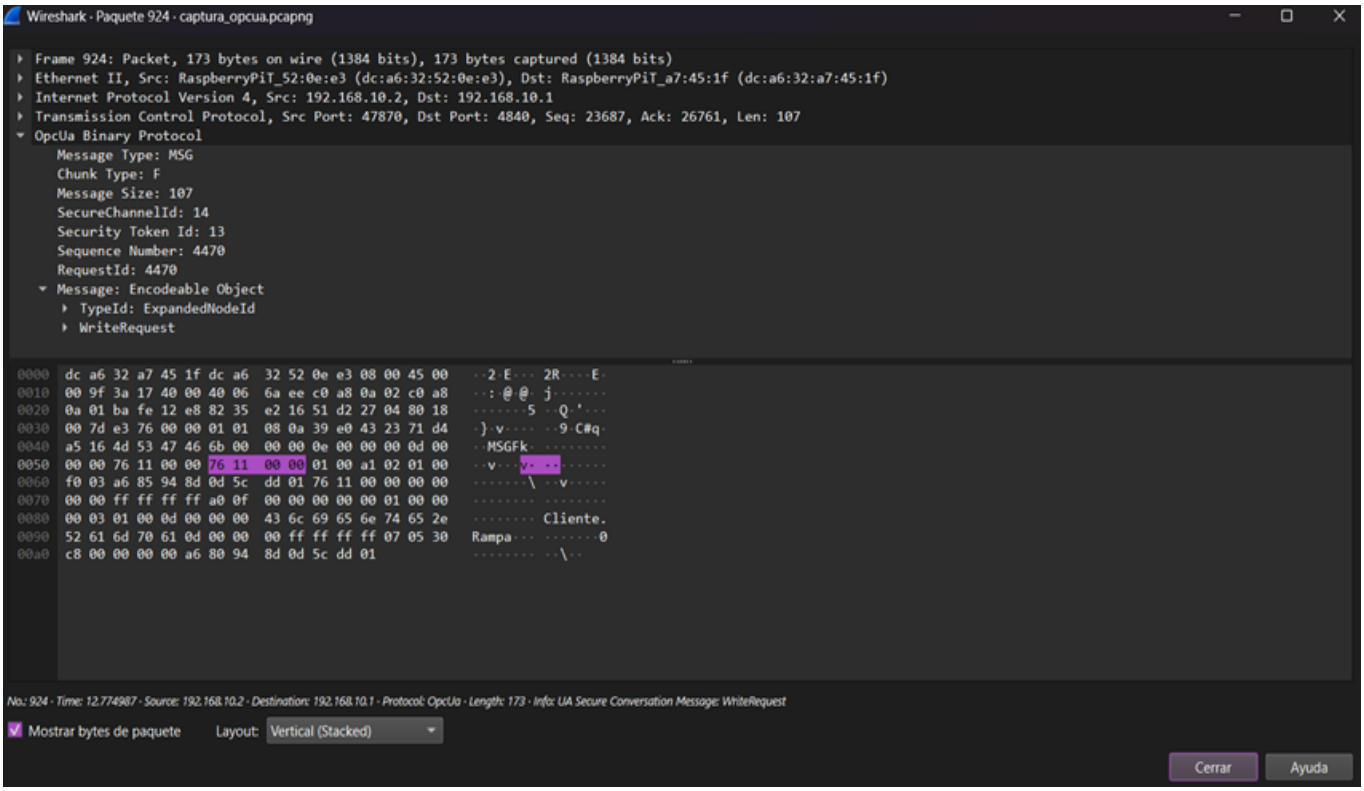
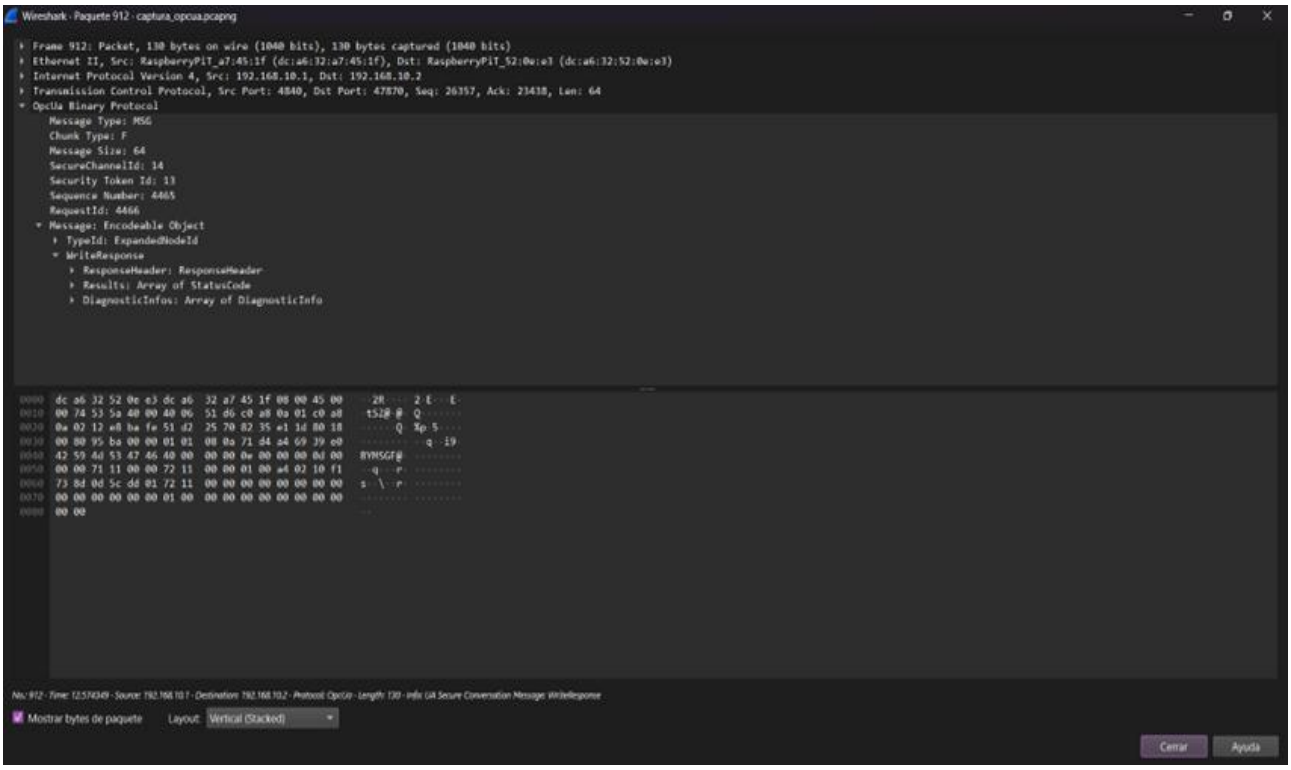


Figura 27. WriteRequest desplegado como Encodeable Object en Wireshark.

8.4 WriteResponse

La respuesta del servidor utiliza el mismo canal seguro y devuelve un `WriteResponse` con `ResponseHeader`, un arreglo de `StatusCode` y diagnósticos. Para asociar de forma inequívoca un request con su response debe utilizarse el `RequestId`, no solamente la posición consecutiva de los frames.



El `WriteResponse` circula en sentido inverso, desde el puerto 4840 del servidor hacia el puerto efímero del cliente. Su función no es volver a transportar el valor completo, sino informar el resultado del servicio solicitado. El arreglo de `StatusCode` permite conocer si la escritura fue aceptada o si ocurrió un error de acceso, tipo o ejecución.

La asociación entre solicitud y respuesta se realiza mediante el `RequestId`. Esto es especialmente importante cuando existen varios servicios simultáneos, ya que dos frames consecutivos no necesariamente forman un par lógico. El análisis del identificador evita atribuir una respuesta a la solicitud equivocada.

Figura 28. WriteResponse desplegado en Wireshark.

8.5 Encapsulamiento

El tráfico capturado presenta el encapsulamiento Ethernet II → IPv4 → TCP → OPC UA. En un WriteRequest típico de 173 bytes se observan 14 bytes de cabecera Ethernet, 20 bytes de IPv4, una cabecera TCP ampliada por opciones y 107 bytes correspondientes al mensaje OPC UA. El valor de proceso útil de `Cliente.Rampa` es un UInt16 de 2 bytes, mientras que el resto de la trama aporta direccionamiento, confiabilidad, sesión, canal y semántica OPC UA.

El encapsulamiento observado permite relacionar el protocolo de aplicación con las capas inferiores utilizadas en el prototipo. Ethernet aporta el direccionamiento MAC del enlace local; IPv4 identifica los hosts 192.168.10.1 y 192.168.10.2; TCP proporciona conexión confiable y ordenada entre el puerto efímero del cliente y el puerto 4840 del servidor; finalmente OPC UA transporta los servicios y el modelo de datos.

Esta estructura explica por qué un valor de proceso pequeño puede generar una trama relativamente grande. El objetivo de OPC UA no es minimizar únicamente la cantidad de bytes, sino transportar información suficiente para identificar servicios, nodos, sesiones, secuencias, estados y, cuando se configura, mecanismos de seguridad.

8.6 Secuencia de establecimiento y cierre observada

El primer tramo del archivo comienza con una sesión OPC UA ya activa, pero dentro de la misma captura se registra posteriormente el cierre de esa conexión y el establecimiento de una nueva sesión desde el inicio. Esto permitió reconstruir la secuencia de transporte y de servicios sin necesidad de una segunda adquisición.

En los frames 2304, 2305 y 2306 se observa el handshake TCP de la nueva conexión mediante `SYN`, `SYN/ACK` y `ACK`. A continuación aparecen `HEL` en el frame 2307 y `ACK` en el frame 2309, correspondientes a la negociación inicial de UA TCP.

La secuencia continúa con `OpenSecureChannel` en los frames 2311–2312, `CreateSession` en 2313–2314 y `ActivateSession` en 2315–2316. Luego se observa `CreateSubscription` en 2321–2322 y la creación de los MonitoredItems en los frames 2323/2326 y 2328/2329. Esta secuencia confirma que el cliente no solamente recupera TCP, sino que establece el contexto lógico necesario para operar por OPC UA.

La captura también contiene el cierre de la sesión anterior: `CloseSessionRequest` en el frame 2297, `CloseSessionResponse` en 2298, el mensaje `CLO` en 2300 y el cierre TCP en los frames siguientes. No se observó un servicio `GetEndpoints` en esta nueva conexión, situación compatible con un cliente que ya conoce el endpoint y se conecta directamente a él. Tampoco se observa ARP en ese tramo, por lo que la resolución IP–MAC ya estaba disponible en caché.

8.7 Eficiencia de trama

Tomando como ejemplo un `WriteRequest` OPC UA de 173 bytes que transporta una variable `UInt16` de 2 bytes, la relación entre dato útil y tamaño total de la trama es:

Eficiencia OPC UA, considerando únicamente la solicitud:

$$\eta = \frac{2}{173} \times 100 \approx 1,16\%$$

Si se incluye además el `WriteResponse` de aproximadamente 130 bytes, el intercambio completo request/response suma:

$$173+130=303 \text{ bytes}$$

Por lo tanto, la eficiencia considerando el intercambio completo resulta:

$$\eta = \frac{2}{303} \times 100 \approx 0,66\%$$

Para realizar una comparación equivalente con MODBus RTU puede considerarse la función 06 (`Write Single Register`), utilizada para escribir una única word de 16 bits. Una trama RTU típica de solicitud está formada por 1 byte de dirección de esclavo, 1 byte de código de función, 2 bytes de dirección de registro, 2 bytes correspondientes al valor del registro y 2 bytes de CRC, resultando en un total de 8 bytes. La respuesta normal posee la misma longitud, ya que el dispositivo responde repitiendo la solicitud.

De esta manera, para MODBus RTU:

Eficiencia considerando únicamente la solicitud:

$$\eta = \frac{2}{8} \times 100 = 25\%$$

Eficiencia considerando solicitud y respuesta:

$$\eta = \frac{2}{16} \times 100 = 12,5\%$$

La comparación muestra que MODBus RTU presenta una mayor eficiencia si se considera únicamente la relación entre bytes de proceso y bytes transmitidos. Frente al 1,16 % obtenido para el `WriteRequest` OPC UA, MODBus RTU alcanza un 25 %. Si se considera el intercambio completo, OPC UA presenta aproximadamente 0,66 %, mientras que MODBus RTU alcanza 12,5 %.

Esta diferencia se debe a que OPC UA transporta una cantidad considerablemente mayor de información asociada al servicio: cabeceras Ethernet, IPv4 y TCP, identificadores de canal seguro, token de seguridad, números de secuencia y solicitud, NodeIds, información de sesión y estructura del servicio OPC UA. En cambio, MODBus RTU utiliza una trama mucho más compacta y orientada directamente a registros.

Por lo tanto, MODBus RTU resulta más eficiente en términos de cantidad de bytes necesarios para transmitir una única word, mientras que OPC UA sacrifica eficiencia de trama a cambio de ofrecer mayores capacidades de interoperabilidad, modelado de información, diagnóstico, navegación, sesiones y seguridad.

Protocolo	Dato útil	Solicitud	Solicitud + respuesta	Eficiencia solicitud	Eficiencia total
OPC UA	2 bytes	173 bytes	303 bytes	1,16 %	0,66 %
MODBus RTU	2 bytes	8 bytes	16 bytes	25 %	12,5 %

9. Configuración de seguridad utilizada

Las pruebas funcionales documentadas en este informe se realizaron con SecurityMode None y autenticación Anonymous. Esta configuración permitió validar el modelo de

información, los servicios de lectura y escritura, las suscripciones, la reconexión y el análisis de tráfico sin incorporar cifrado durante esta etapa experimental.

10. Diseño lógico, puesta en marcha y trazabilidad

La secuencia de puesta en marcha resume el procedimiento reproducible utilizado para preparar las microSD, configurar la red, instalar dependencias, iniciar servidor y cliente y validar el tráfico.

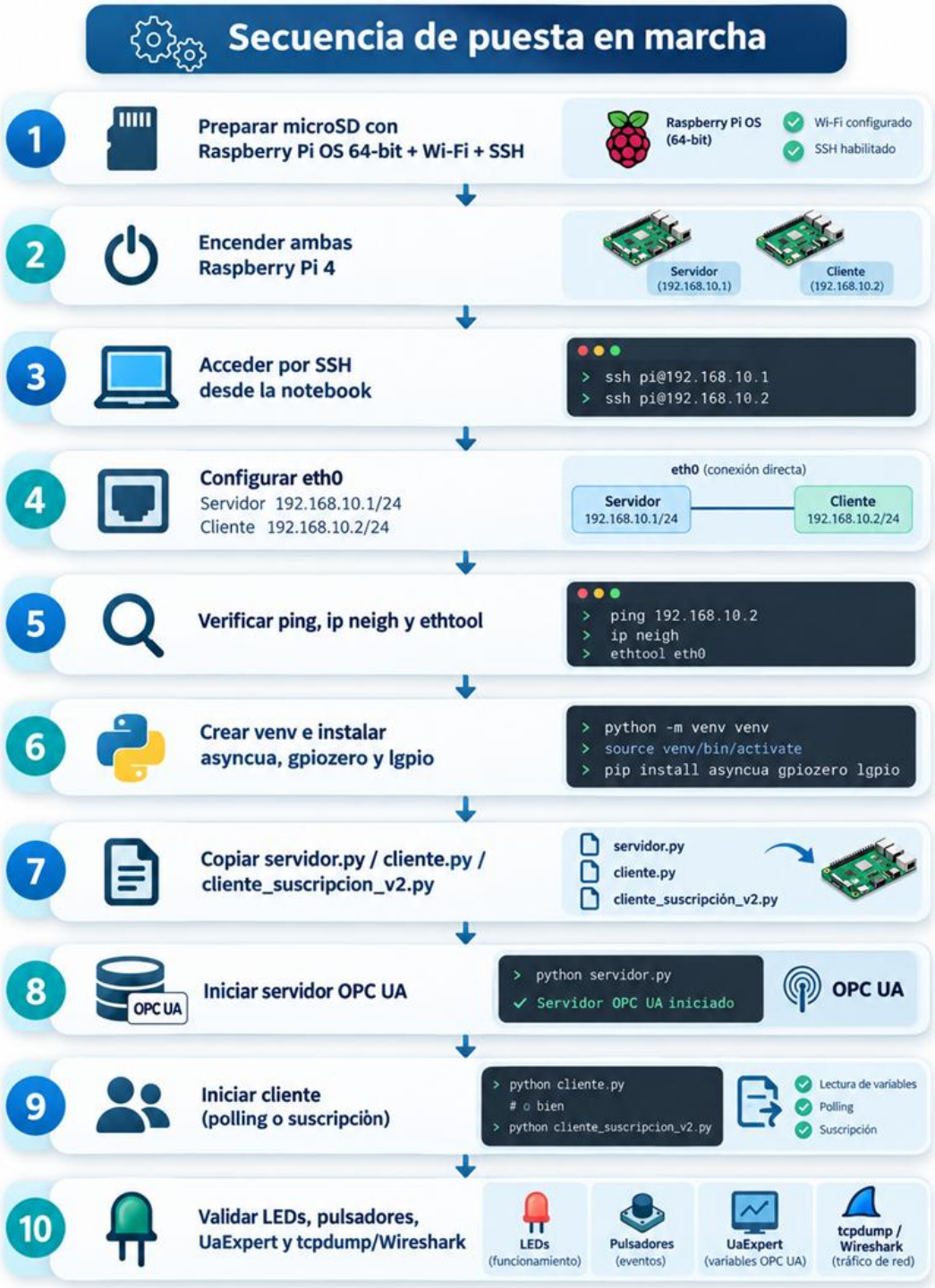


Figura 30. Secuencia completa de puesta en marcha del prototipo.

10.1 Diagrama de flujo y trazabilidad del servidor

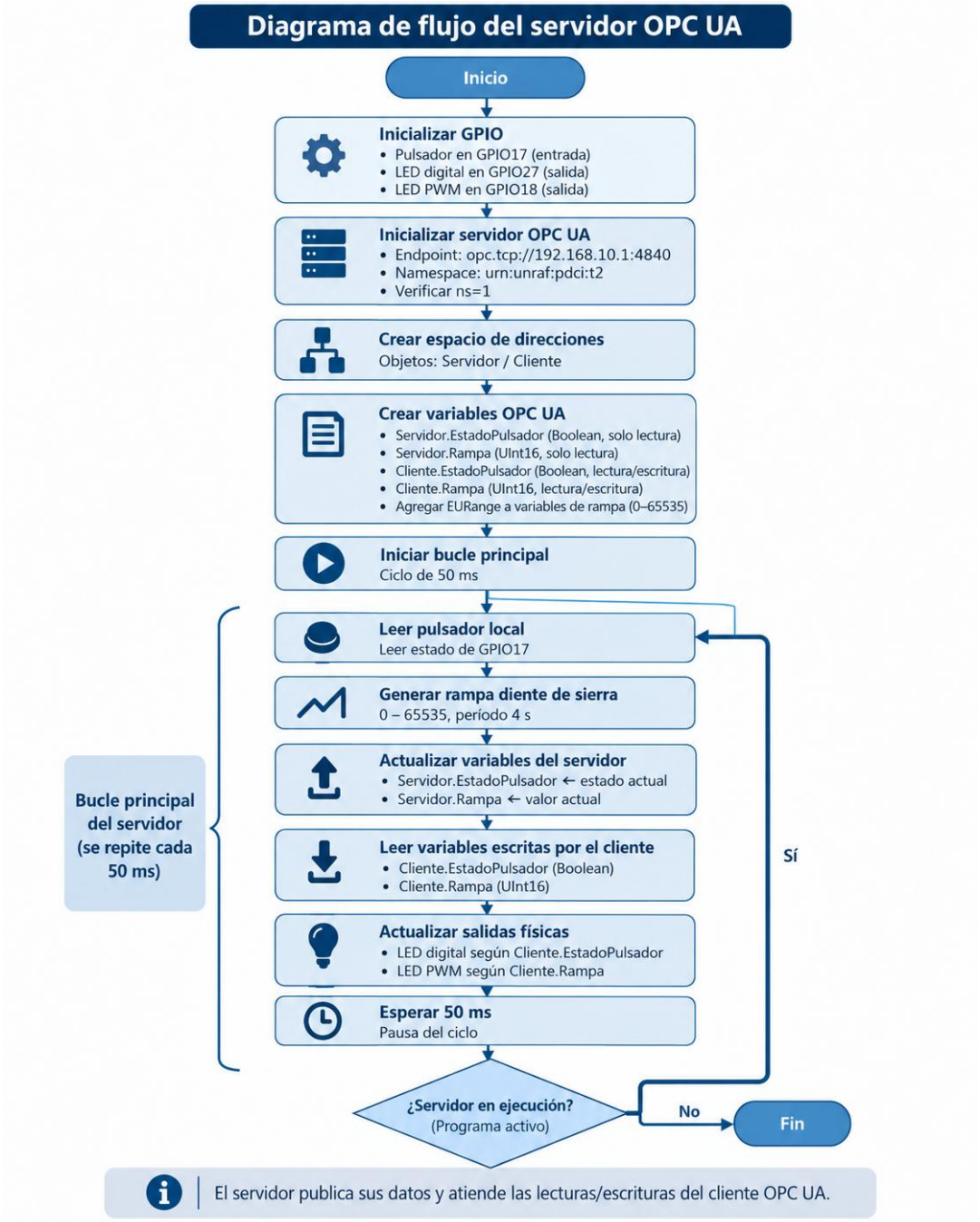
El servidor inicializa GPIO y OPC UA, crea el modelo de información, genera su rampa local, publica el estado del pulsador, lee las variables escritas por el cliente y actualiza sus LEDs. El ciclo principal se ejecuta aproximadamente cada 50 ms.

```
# Síntesis del ciclo del servidor
estado_local = bool(pulsador.is_pressed)
fase = ((ahora - inicio_rampa) % 4.0) / 4.0
valor_rampa_local = min(65535, int(fase * 65536))

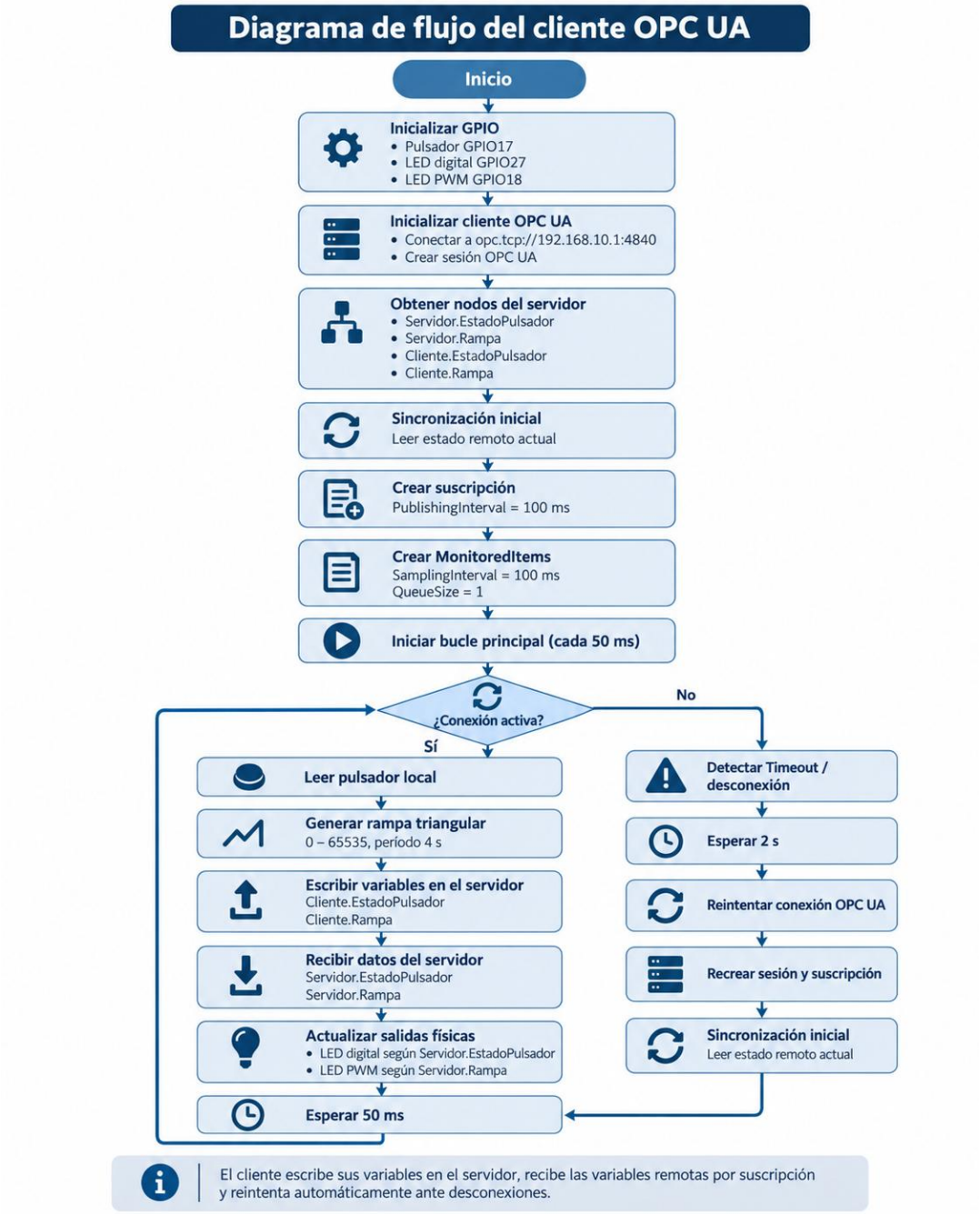
await estado_servidor.write_value(...)
await rampa_servidor.write_value(...)

estado_remoto = bool(await estado_cliente.read_value())
valor_rampa_remota = int(await rampa_cliente.read_value())

led_digital.value = estado_remoto
led_pwm.value = valor_rampa_remota / 65535.0
```


Diagrama de flujo del servidor OPC UA
Inicio
Inicializar GPIO
• Pulsador en GPIO17 (entrada)
• LED digital en GPIO27 (salida)
• LED PWM en GPIO18 (salida)
Inicializar servidor OPC UA
• Endpoint: opc.tcp://192.168.10.1:4840
• Namespace: urn:unraf:pdci:t2
• Verificar ns=1
Crear espacio de direcciones
Objetos: Servidor / Cliente
Crear variables OPC UA
• Servidor.EstadoPulsador (Boolean, solo lectura)
• Servidor.Rampa (UInt16, solo lectura)
• Cliente.EstadoPulsador (Boolean, lectura/escritura)
• Cliente.Rampa (UInt16, lectura/escritura)
• Agregar EURange a variables de rampa (0–65535)
Iniciar bucle principal
Ciclo de 50 ms
Bucle principal del servidor (se repite cada 50 ms)
Leer pulsador local
Leer estado de GPIO17
Generar rampa diente de sierra
0 – 65535, período 4 s
Actualizar variables del servidor
• Servidor.EstadoPulsador ← estado actual
• Servidor.Rampa ← valor actual
Leer variables escritas por el cliente
• Cliente.EstadoPulsador (Boolean)
• Cliente.Rampa (UInt16)
Actualizar salidas físicas
• LED digital según Cliente.EstadoPulsador
• LED PWM según Cliente.Rampa
Esperar 50 ms
Pausa del ciclo
¿Servidor en ejecución?
(Programa activo)
Sí
No
Fin
El servidor publica sus datos y atiende las lecturas/escrituras del cliente OPC UA.

10.2 Diagrama de flujo y trazabilidad del cliente por suscripción



El programa `cliente_suscripcion_v2.py` implementa la lógica representada en el diagrama de flujo del cliente OPC UA. Su funcionamiento se divide en distintas etapas que permiten establecer la comunicación con el servidor, intercambiar datos, actualizar las salidas físicas y recuperarse automáticamente ante una pérdida del enlace.

En primer lugar, el programa inicializa los elementos físicos conectados a la Raspberry Pi 2: el pulsador local, el LED digital y el LED PWM. A continuación, se crea el cliente OPC UA y se establece la conexión con el endpoint del servidor:

```
opc.tcp://192.168.10.1:4840
```

Una vez establecida la comunicación, el cliente obtiene las referencias a los nodos OPC UA utilizados durante el intercambio:

```
ns=1;s=Servidor.EstadoPulsador
ns=1;s=Servidor.Rampa
ns=1;s=Cliente.EstadoPulsador
ns=1;s=Cliente.Rampa
```

Antes de crear la suscripción, el programa realiza una sincronización inicial. En esta etapa se leen una vez los valores actuales de `Servidor.EstadoPulsador` y `Servidor.Rampa`, y estos se aplican directamente al LED digital y al LED PWM del cliente. Esta sincronización permite que, después de una conexión o reconexión, las salidas físicas reflejen inmediatamente el estado real de las variables remotas.

Luego se crea una suscripción OPC UA con un `PublishingInterval` de 100 ms. Sobre esta suscripción se crean dos MonitoredItems: uno para `Servidor.EstadoPulsador` y otro para `Servidor.Rampa`, utilizando un `SamplingInterval` de 100 ms y `QueueSize = 1`.

La versión `cliente_suscripcion_v2.py` utiliza el método `subscribe_data_change()` para crear los MonitoredItems. De esta manera, la librería `asyncua` asigna correctamente identificadores distintos a cada elemento monitoreado. Esta corrección fue necesaria porque en una versión anterior el estado del pulsador del servidor no generaba correctamente la notificación en el cliente.

Cuando el servidor detecta un cambio en alguno de los nodos monitoreados, el cliente recibe una notificación mediante el manejador de cambio de datos. A partir de esta notificación, se actualizan las variables internas y las salidas físicas. `Servidor.EstadoPulsador` controla el LED digital del cliente, mientras que `Servidor.Rampa` controla el brillo del LED PWM.

Paralelamente, el cliente genera sus propios datos locales. El estado del pulsador de la Raspberry Pi 2 se escribe en `Cliente.EstadoPulsador`, mientras que la rampa triangular generada localmente se escribe periódicamente en `Cliente.Rampa`. De esta manera se produce un intercambio bidireccional de información: el cliente recibe datos del servidor mediante la suscripción y, al mismo tiempo, escribe sus propias variables en el espacio de direcciones del servidor.

El programa también incorpora una lógica de reconexión automática. Si la comunicación Ethernet se interrumpe, el cliente detecta un error de timeout y pierde la sesión activa. Luego espera aproximadamente 2 segundos y vuelve a intentar la conexión.

Cuando la comunicación se restablece, el cliente vuelve a conectarse al servidor, obtiene nuevamente las referencias a los nodos, realiza la sincronización inicial y recrea la suscripción y los MonitoredItems. Esto permite recuperar automáticamente el funcionamiento del sistema sin intervención manual.

En síntesis, `cliente_suscripcion_v2.py` implementa las funciones de conexión, obtención de nodos, sincronización inicial, creación de la suscripción, monitoreo de variables, escritura de datos locales, recepción de notificaciones, actualización de salidas físicas y reconexión automática ante una pérdida del enlace.

Fragmento principal de creación de la suscripción y MonitoredItems:

```
subscription = await client.create_subscription(100, manejador)

handle_estado = await subscription.subscribe_data_change(
    estado_servidor, sampling_interval=100, queuesize=1
)
handle_rampa = await subscription.subscribe_data_change(
```

```
        rampa_servidor, sampling_interval=100, queuesize=1
    )
```

11. Resultados y conclusiones

11.1 Resultados obtenidos

- Enlace Ethernet punto a punto operativo a 1000 Mb/s Full Duplex.
- Conectividad IPv4 bidireccional con 0 % de pérdida en las pruebas realizadas.
- Servidor OPC UA operativo en el puerto 4840 con namespace ns=1.
- Modelo de información con los cuatro NodeIds exigidos.
- Intercambio bidireccional físico verificado mediante pulsadores y LEDs.
- Modo polling funcional con registro de SourceTimestamp.
- Modo Subscription/MonitoredItems funcional con parámetros de 100 ms y QueueSize 1.
- Recuperación automática de comunicación y resincronización después de una desconexión Ethernet.
- Captura real de tráfico OPC UA y análisis de mensajes Publish y Write en Wireshark.
- UaExpert utilizado como cliente OPC UA remoto con la Raspberry cliente apagada, con escritura efectiva sobre Cliente.EstadoPulsador y Cliente.Rampa.
- Tendencias de Servidor.Rampa y EstadoPulsador verificadas en UaExpert.
- Escritura sobre variable de solo lectura rechazada por el servidor con indicación de acceso denegado.
- Durante la pérdida de Ethernet se verificó que el LED PWM remoto queda congelado mientras el generador local continúa ejecutándose.
- La captura de Wireshark permitió verificar handshake TCP, HEL/ACK, OpenSecureChannel, CreateSession, ActivateSession, CreateSubscription, CreateMonitoredItems, servicios Publish/Write y cierre de sesión.

11.2 Conclusión

El trabajo permitió implementar y validar una comunicación OPC UA cliente-servidor sobre dos Raspberry Pi 4 conectadas directamente por Ethernet. La configuración de direcciones estáticas, la verificación mediante ping, `ip neigh` y `ethtool`, y la captura sobre `eth0` permitieron relacionar la aplicación OPC UA con la infraestructura de red utilizada. El enlace operó a 1000 Mb/s Full Duplex y el tráfico entre ambas placas se mantuvo dentro de la red 192.168.10.0/24.

A nivel de aplicación se construyó un Address Space propio con NodeIds fijos para las variables de servidor y cliente. La prueba con UaExpert confirmó la posibilidad de navegar el modelo, visualizar valores, StatusCode y timestamps, observar las tendencias de la rampa y el pulsador y escribir remotamente sobre los nodos habilitados. El rechazo de la escritura sobre una variable de solo lectura mostró además que los permisos definidos en el servidor tienen efecto sobre las operaciones del cliente.

El intercambio físico demostró comunicación bidireccional: cada Raspberry genera su pulsador y su rampa local, mientras sus LEDs responden a información proveniente de la otra placa. Se comprobaron dos mecanismos de acceso a datos. En polling el cliente inicia lecturas periódicas; en suscripción el servidor supervisa MonitoredItems y entrega cambios mediante PublishResponse sobre solicitudes Publish mantenidas por el cliente. La corrección del problema de ClientHandle y la recreación de la suscripción después de una desconexión permitieron verificar aspectos reales de implementación y recuperación.

La prueba de pérdida de Ethernet mostró que el procesamiento local continúa aun cuando se interrumpe la información remota: el generador local sigue funcionando y el LED PWM dependiente de la variable remota conserva el último valor recibido. Al restablecer el enlace, la reconexión automática, la obtención de nodos, la sincronización inicial y la recreación de MonitoredItems restituyen el funcionamiento sin intervención manual.

Finalmente, el análisis de Wireshark permitió observar tanto el tráfico de operación como la secuencia de establecimiento OPC UA: handshake TCP, HEL/ACK, OpenSecureChannel, CreateSession, ActivateSession, CreateSubscription y CreateMonitoredItems, además de mensajes Publish y Write y el cierre de sesión. La comparación de eficiencia con MODBus RTU evidenció que OPC UA introduce un overhead considerable, pero a cambio aporta un modelo de información navegable, servicios estructurados, interoperabilidad, contexto de sesión y capacidades de seguridad. En conjunto, el prototipo permitió integrar conceptos

de red, programación, instrumentación y análisis de protocolos en una implementación funcional y verificable.

12. Referencias bibliográficas

OPC Foundation. (2025). OPC Unified Architecture – Part 1: Overview and concepts (OPC 10000-1, Version 1.05.06). https://reference.opcfoundation.org/specs/OPC-10000-1/full

OPC Foundation. (2025). OPC Unified Architecture – Part 3: Address Space Model. https://reference.opcfoundation.org/

OPC Foundation. (2025). OPC Unified Architecture – Part 4: Services. https://reference.opcfoundation.org/

OPC Foundation. (2025). OPC Unified Architecture – Part 2: Security Model. https://reference.opcfoundation.org/

Plummer, D. C. (1982). An Ethernet Address Resolution Protocol (RFC 826). RFC Editor. https://www.rfc-editor.org/info/rfc826/

Modbus Organization. (2012). MODBUS application protocol specification V1.1b3. https://www.modbus.org/file/secure/modbusprotocolspecification.pdf